

AI 500

Foundations of

Artificial Intelligence

L7: Support Vector Machines

By Murtaza Taj



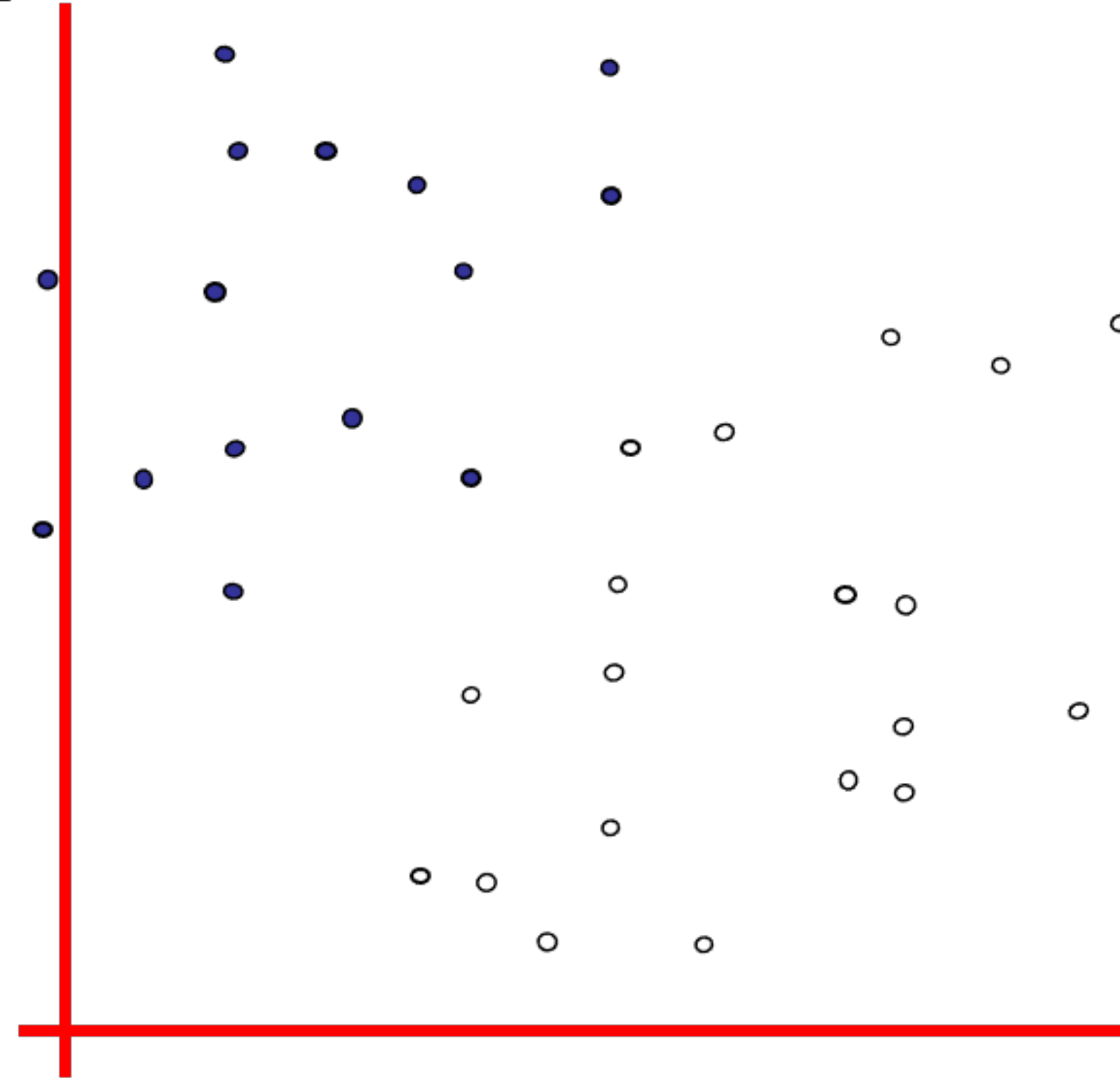
Reminder: **Mid-terms**

- Date: Sat 25th Oct 2025
- Venue: SDSB B1& B2
- Time: 2:00pm to 5:00pm
- Syllabus: L1-L7 (excluding Causal Inference)
- Closed book/closed notes
- Single 2 sided hand written A4 sized cheat sheet is allowed

Support Vector Machines

Linear Classifier

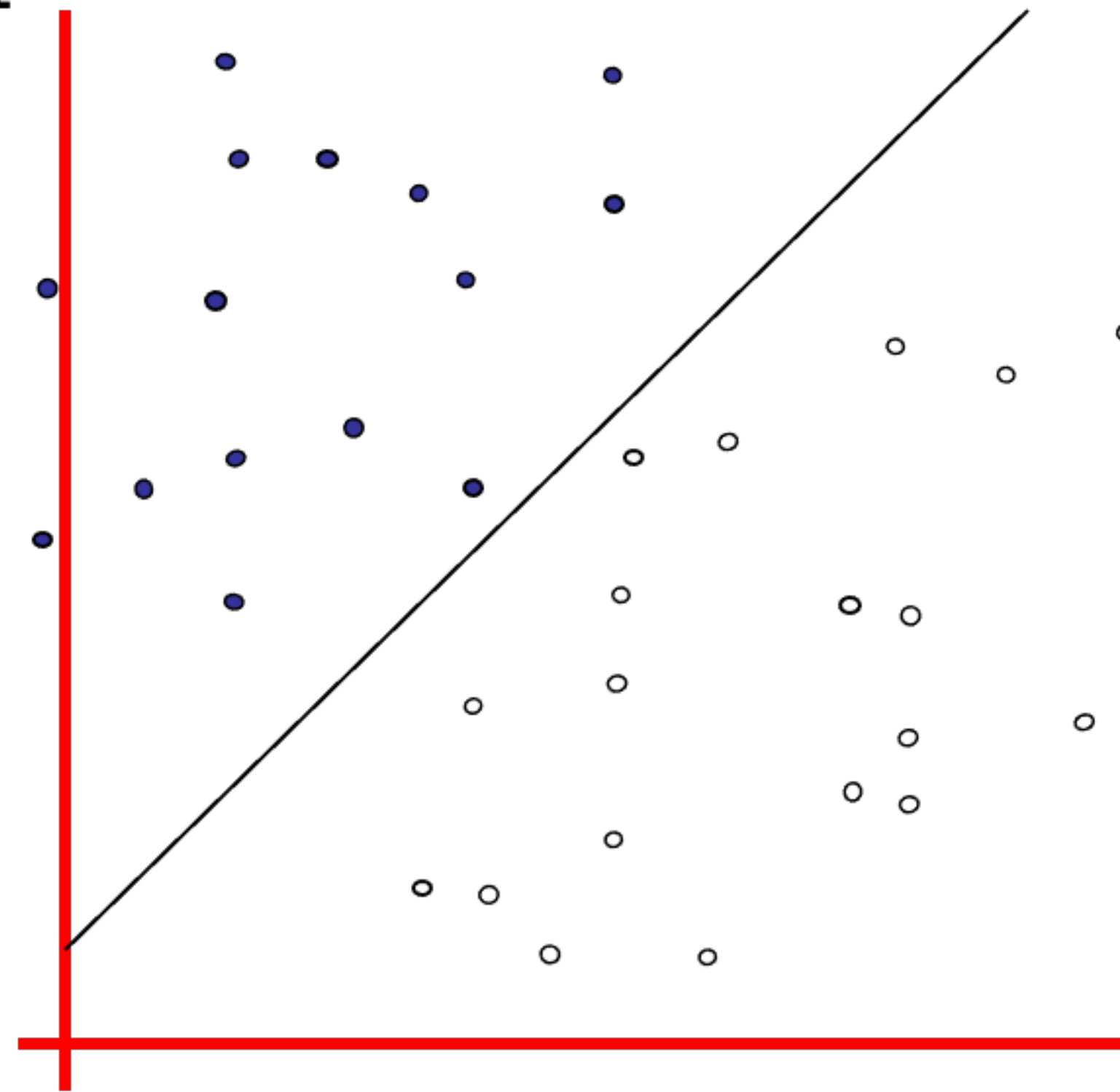
- denotes +1
- denotes -1



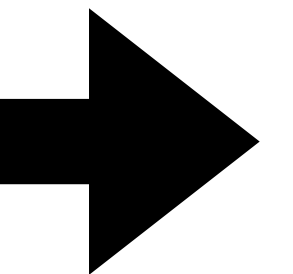
How would you
classify this data?

Linear Classifier

- denotes +1
- denotes -1



How would you
classify this data?

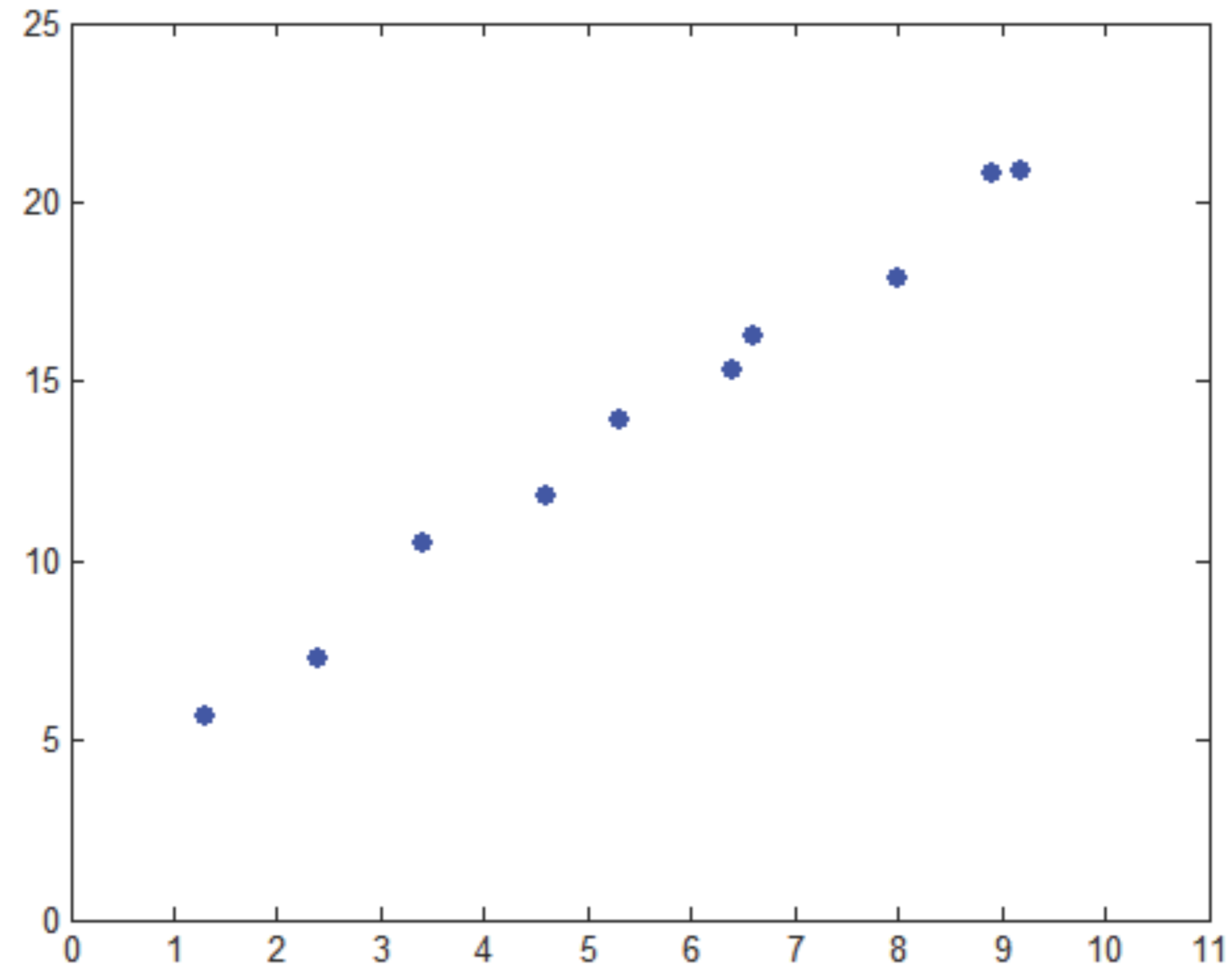


Linear Regression

Linear Fitting: Least Square Error Solutions

- ▶ Let us first consider the 'simpler' problem of fitting a line to a set of data points...

x	y
1.3	5.7
2.4	7.3
3.4	10.5
4.6	11.8
5.3	13.9
6.6	16.3
6.4	15.3
8.0	17.9
8.9	20.8
9.2	20.9



- ▶ Equation of best fit line ?

Linear Fitting: Least Square Error Solutions

- ▶ Step 1: Identify the model
 - ▶ Equation of line: $y = mx + c$

Linear Fitting: Least Square Error Solutions

- ▶ Step 1: Identify the model
 - ▶ Equation of line: $y = mx + c$
- ▶ Step 2: Set up an error term which will give the goodness of every point with respect to the (unknown) model
 - ▶ Error induced by i^{th} point: $e^{(i)} = (t^{(i)} - mx^{(i)} - c)^2$
 - ▶ Error for whole data:
$$E = \sum_i (t^{(i)} - y^{(i)})^2$$
$$E = \sum_i (t^{(i)} - mx^{(i)} - c)^2$$

Linear Fitting: Least Square Error Solutions

- ▶ Step 1: Identify the model
 - ▶ Equation of line: $y = mx + c$
- ▶ Step 2: Set up an error term which will give the goodness of every point with respect to the (unknown) model
 - ▶ Error induced by i^{th} point: $e^{(i)} = (t^{(i)} - mx^{(i)} - c)^2$
 - ▶ Error for whole data:
$$E = \sum (t^{(i)} - y^{(i)})^2$$
$$E = \sum_i (t^{(i)} - mx^{(i)} - c)^2$$
- ▶ Step 3: Differentiate Error w.r.t. parameters, put equal to zero and solve for minimum point
- ▶ In other words, find the set of parameters m and c for which the error term E is minimized

Linear Fitting: Least Square Error Solutions

$$E = \sum_i (t^{(i)} - mx^{(i)} - c)^2$$

x	t
1.3	5.7
2.4	7.3
3.4	10.5
4.6	11.8
5.3	13.9
6.6	16.3
6.4	15.3
8.0	17.9
8.9	20.8
9.2	20.9

Linear Fitting: Least Square Error Solutions

$$E = \sum_i (t^{(i)} - mx^{(i)} - c)^2$$

$$\frac{\partial E}{\partial m} = - \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)}$$

x	t
1.3	5.7
2.4	7.3
3.4	10.5
4.6	11.8
5.3	13.9
6.6	16.3
6.4	15.3
8.0	17.9
8.9	20.8
9.2	20.9

Linear Fitting: Least Square Error Solutions

$$E = \sum_i (t^{(i)} - mx^{(i)} - c)^2$$

$$\frac{\partial E}{\partial m} = - \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)}$$

$$\frac{\partial E}{\partial c} = - \sum_i (t^{(i)} - mx^{(i)} + c)$$

x	t
1.3	5.7
2.4	7.3
3.4	10.5
4.6	11.8
5.3	13.9
6.6	16.3
6.4	15.3
8.0	17.9
8.9	20.8
9.2	20.9

Linear Fitting: Least Square Error Solutions

$$\frac{\partial E}{\partial m} = - \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)}$$

$$- \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)} = 0$$

$$- \sum_i t^{(i)}x^{(i)} + m \sum_i (x^{(i)})^2 + c \sum_i x^{(i)} = 0$$

$$\sum_i m(x^{(i)})^2 + \sum_i cx^{(i)} = \sum_i t^{(i)}x^{(i)}$$

$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \\ - & - \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)}x^{(i)} \\ - \end{bmatrix}$$

Linear Fitting: Least Square Error Solutions

$$\frac{\partial E}{\partial m} = - \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)}$$

$$- \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)} = 0$$

$$- \sum_i t^{(i)}x^{(i)} + m \sum_i (x^{(i)})^2 + c \sum_i x^{(i)} = 0$$

$$\sum_i m(x^{(i)})^2 + \sum_i cx^{(i)} = \sum_i t^{(i)}x^{(i)}$$

$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)}x^{(i)} \end{bmatrix}$$

$1 \times 2 \qquad 2 \times 1 \qquad 1 \times 1$

$$\frac{\partial E}{\partial c} = - \sum_i (t^{(i)} - mx^{(i)} + c)$$

$$- \sum_i (t^{(i)} - mx^{(i)} + c) = 0$$

$$- \sum_i t^{(i)} + m \sum_i x^{(i)} + \sum_i c = 0$$

$$m \sum_i x^{(i)} + c \sum_i 1 = \sum_i t^{(i)}$$

$$\begin{bmatrix} m \sum_i x^{(i)} & c \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)} \end{bmatrix}$$

$1 \times 2 \qquad 2 \times 1 \qquad 1 \times 1$

Linear Fitting: Least Square Error Solutions

$$\frac{\partial E}{\partial m} = - \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)}$$

$$- \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)} = 0$$

$$- \sum_i t^{(i)}x^{(i)} + m \sum_i (x^{(i)})^2 + c \sum_i x^{(i)} = 0$$

$$\sum_i m(x^{(i)})^2 + \sum_i cx^{(i)} = \sum_i t^{(i)}x^{(i)}$$

$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)}x^{(i)} \end{bmatrix}$$

$$\frac{\partial E}{\partial c} = - \sum_i (t^{(i)} - mx^{(i)} + c)$$

$$- \sum_i (t^{(i)} - mx^{(i)} + c) = 0$$

$$- \sum_i t^{(i)} + m \sum_i x^{(i)} + \sum_i c = 0$$

$$m \sum_i x^{(i)} + c \sum_i 1 = \sum_i t^{(i)}$$

$$\begin{bmatrix} m \sum_i x^{(i)} & c \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)} \end{bmatrix}$$

$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \\ \sum_i x^{(i)} & \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i t^{(i)}x^{(i)} \\ \sum_i t^{(i)} \end{bmatrix}$$

$2 \times 2 \qquad 2 \times 1 \qquad 2 \times 1$

$$\mathbf{Ax} = \mathbf{b}$$

Linear Fitting: Least Square Error Solutions

$$E = \sum_i (t^{(i)} - mx^{(i)} - c)^2$$

$$\frac{\partial E}{\partial m} = - \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)}$$

$$\frac{\partial E}{\partial c} = - \sum_i (t^{(i)} - mx^{(i)} - c)$$

$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \\ \sum_i x^{(i)} & \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i x^{(i)} t^{(i)} \\ \sum_i t^{(i)} \end{bmatrix}$$

$$\mathbf{Ax} = \mathbf{b}$$

x	t
1.3	5.7
2.4	7.3
3.4	10.5
4.6	11.8
5.3	13.9
6.6	16.3
6.4	15.3
8.0	17.9
8.9	20.8
9.2	20.9

Linear Fitting: Least Square Error Solutions

$$E = \sum_i (t^{(i)} - mx^{(i)} - c)^2$$

$$\frac{\partial E}{\partial m} = - \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)}$$

$$\frac{\partial E}{\partial c} = - \sum_i (t^{(i)} - mx^{(i)} - c)$$

$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \\ \sum_i x^{(i)} & \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i x^{(i)} t^{(i)} \\ \sum_i t^{(i)} \end{bmatrix}$$

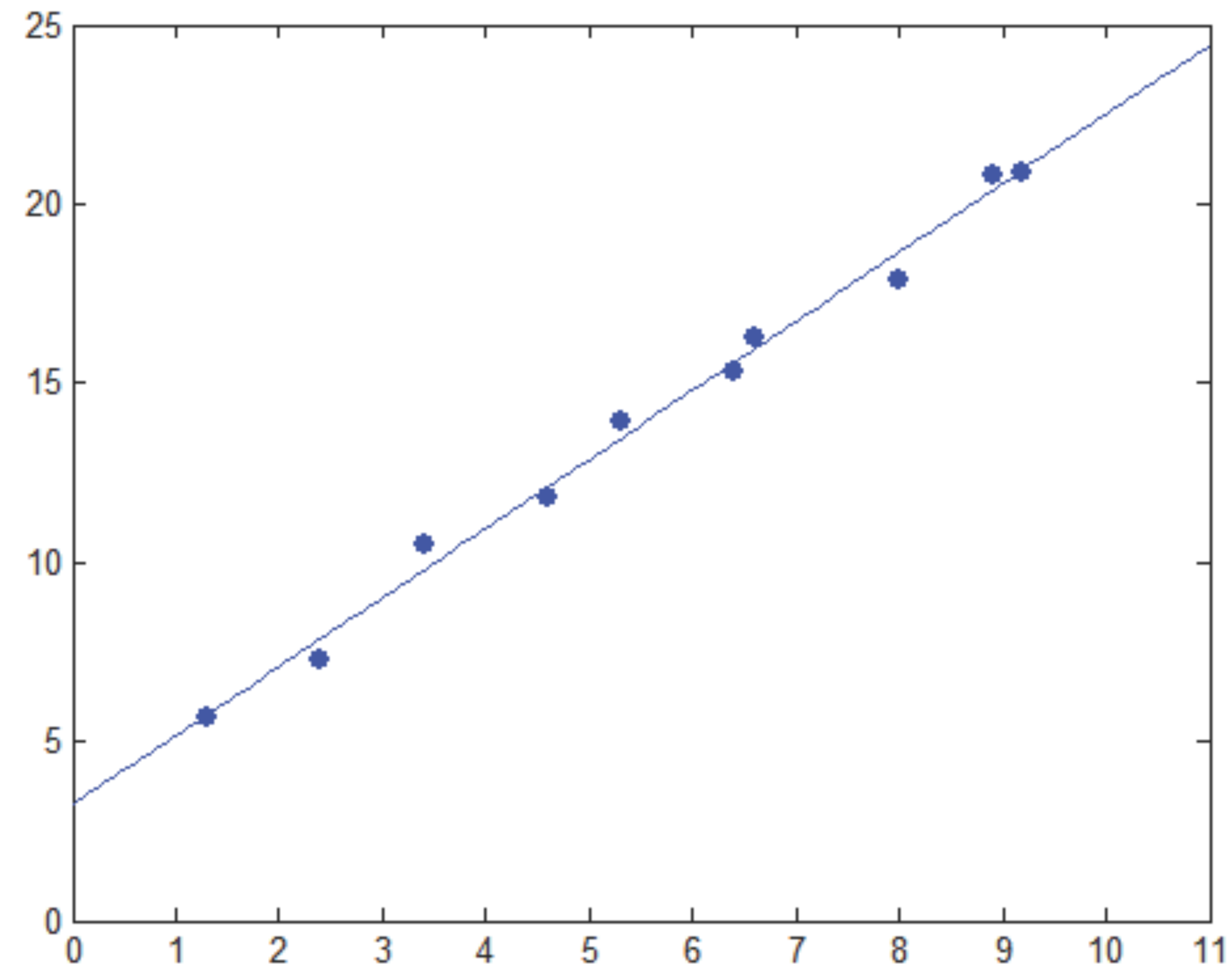
$$\mathbf{Ax} = \mathbf{b}$$

$$\begin{bmatrix} 380.63 & 56.1 \\ 56.1 & 10 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} 914.68 \\ 140.4 \end{bmatrix}$$

Solution: $m = 1.9274$ $c = 3.227$

x	t
1.3	5.7
2.4	7.3
3.4	10.5
4.6	11.8
5.3	13.9
6.6	16.3
6.4	15.3
8.0	17.9
8.9	20.8
9.2	20.9

Linear Fitting: Least Square Error Solutions



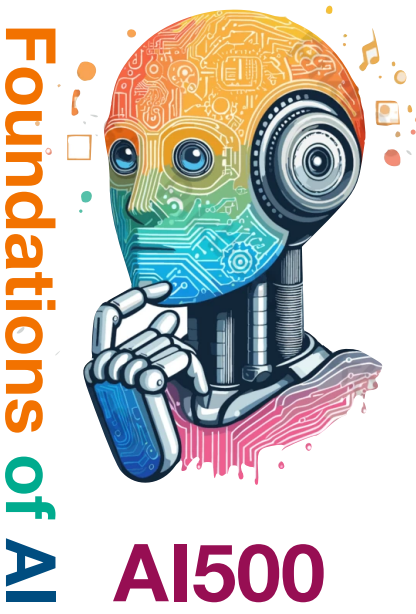
Linear Fitting: Least Square Error Solutions

$$y = mx + c$$

$$\mathbf{w} = \{w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{x_1, x_2, \dots\}^T$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} + w_0 = \sum_j w_j x_j^{(i)} + w_0$$



Linear Fitting: Least Square Error Solutions



$$y = mx + c$$

$$E = \sum_i (t^{(i)} - mx^{(i)} - c)^2$$

$$\mathbf{w} = \{w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{x_1, x_2, \dots\}^T$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} + w_0 = \sum_j w_j x_j^{(i)} + w_0$$

$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{1, x_1, x_2, \dots\}^T$$

$$E = \sum_i (t^{(i)} - y^{(i)})^2 = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

Linear Fitting: Least Square Error Solutions

$$y = mx + c$$

$$E = \sum_i (t^{(i)} - mx^{(i)} - c)^2$$

$$\frac{\partial E}{\partial m} = - \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)}$$

$$\frac{\partial E}{\partial c} = - \sum_i (t^{(i)} - mx^{(i)} + c)$$

$$\mathbf{w} = \{w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{x_1, x_2, \dots\}^T$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} + w_0 = \sum_j w_j x_j^{(i)} + w_0$$

$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{1, x_1, x_2, \dots\}^T$$

$$E = \sum_i (t^{(i)} - y^{(i)})^2 = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

$$\frac{\partial E}{\partial w_j} = - \sum_i (t^{(i)} - y^{(i)})x^{(i)}$$

Linear Fitting: Least Square Error Solutions



$$\mathbf{w} = \{w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{x_1, x_2, \dots\}^T$$

$$y = mx + c$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} + w_0 = \sum_j w_j x_j^{(i)} + w_0$$

$$E = \sum_i (t^{(i)} - mx^{(i)} + c)^2$$

$$E = \sum_i (t^{(i)} - y^{(i)})^2 = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

$$\frac{\partial E}{\partial m} = - \sum_i (t^{(i)} - mx^{(i)} - c)x^{(i)}$$

$$\frac{\partial E}{\partial w_j} = - \sum_i (t^{(i)} - y^{(i)})x_j^{(i)}$$

$$\frac{\partial E}{\partial c} = - \sum_i (t^{(i)} - mx^{(i)} - c)$$

$$\begin{bmatrix} \sum_i (x^{(i)})^2 & \sum_i x^{(i)} \\ \sum_i x^{(i)} & \sum_i 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum_i x^{(i)} t^{(i)} \\ \sum_i t^{(i)} \end{bmatrix}$$

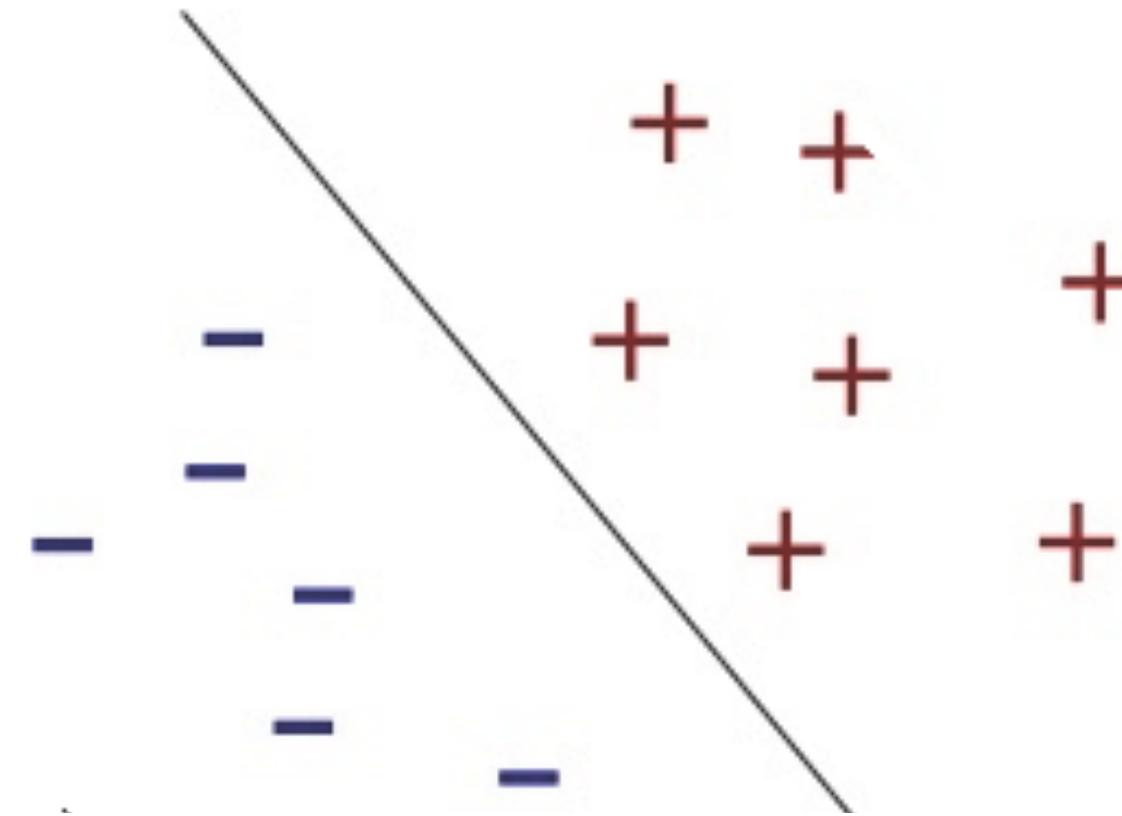
Linear Classifier

$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{1, x_1, x_2, \dots\}^T$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} = \sum_j w_j x_j^{(i)}$$

$$E = \sum_i (t^{(i)} - y^{(i)})^2 = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$



Linear Classifier

$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{1, x_1, x_2, \dots\}^T$$

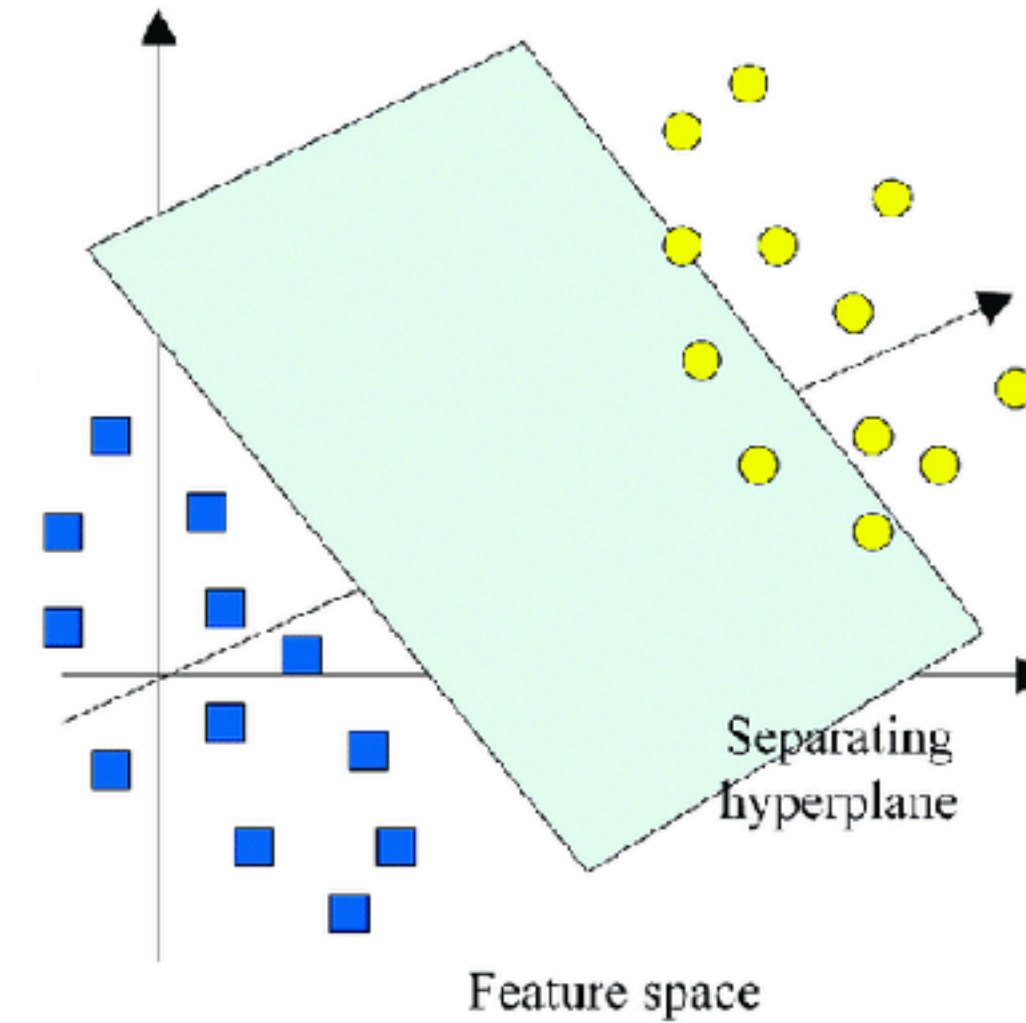
$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} = \sum_j w_j x_j^{(i)}$$

$$t^{(i)} \in \{-1, +1\}$$

$$E = \sum_i (t^{(i)} - y^{(i)})^2 = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

$$\frac{\partial E}{\partial w_j} = - \sum_i (t^{(i)} - y^{(i)}) x_j^{(i)}$$

$$\begin{cases} +1 & y > 0 \\ -1 & y \leq 0 \end{cases}$$



Lbs	Width	Height	y
1	3	2	Insect
1	4	3	Insect
2	11	12	Bird
2	14	9	Bird

▶ $\mathbf{w} = \{10, -1, -1\}$

▶ $1 \times 10 + (-1) \times 3 + (-1) \times 2 > 0$

Linear Regression



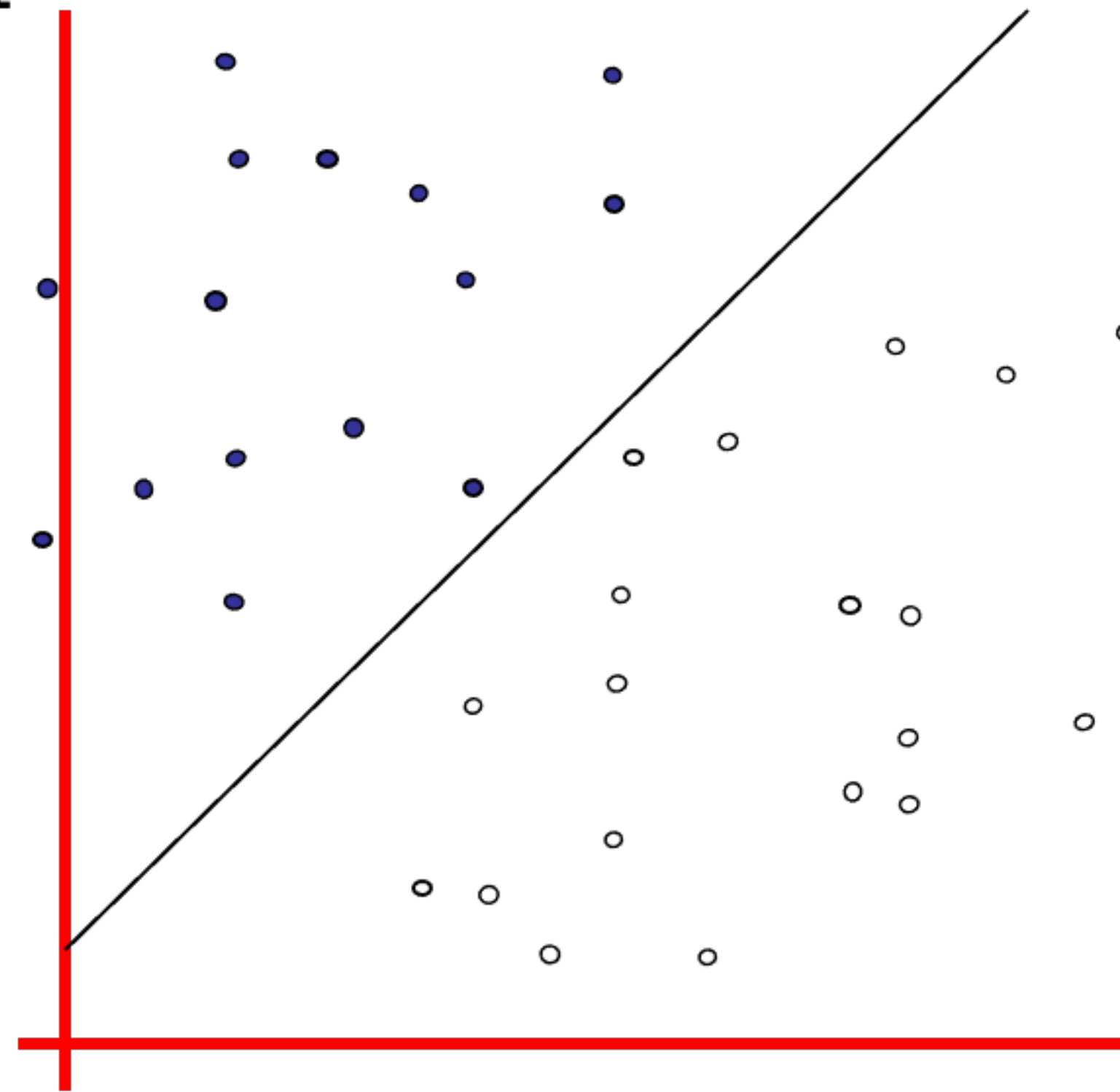
```
train.py  LinearRegression.py 4 •
LinearRegression.py > LinearRegression > fit
1  import numpy as np
2
3
4  class LinearRegression:
5
6      def __init__(self, lr = 0.001, n_iters=1000):
7          self.lr = lr
8          self.n_iters = n_iters
9
10         for _ in range(self.n_iters):
11             y_pred = np.dot(X, self.weights) + self.bias
12
13         def fit(self, X, y):
14             n_samples, n_features = X.shape
15             self.weights = np.zeros(n_features)
16             self.bias = 0
17
18             y_pred = np.dot(X, self.weights) + self.bias
19
20             dw = (1/n_samples) * np.dot(X.T, (y_pred-y))
21             db = (1/n_samples) * np.sum(y_pred-y)
22
23             self.weights = self.weights - self.lr * dw
24             self.bias = self.bias - self.lr * db
25
26         def predict(self, X):
27             y_pred = np.dot(X, self.weights) + self.bias
28             return y_pred
```

```
train.py > ...
1  import numpy as np
2  from sklearn.model_selection import train_test_split
3  from sklearn import datasets
4  import matplotlib.pyplot as plt
5  from LinearRegression import LinearRegression
6
13
14  reg = LinearRegression()          samples=100, n_features=1, noise=20, random_state=4
15  reg.fit(X_train,y_train)          train_test_split(X, y, test_size=0.2, random_state=1234)
16  predictions = reg.predict(X_test)
17
11  plt.scatter(X[:, 0], y, color = "b", marker = "o", s = 30)
12  plt.show()
```

```
16  predictions = reg.predict(X_test)
17
18  def mse(y_test, predictions):
19      return np.mean((y_test-predictions)**2)
20
21  mse = mse(y_test, predictions)
22  print(mse)
23
24  y_pred_line = reg.predict(X)
25  cmap = plt.get_cmap('viridis')
26  fig = plt.figure(figsize=(8,6))
27  m1 = plt.scatter(X_train, y_train, color=cmap(0.9), s=10)
28  m2 = plt.scatter(X_test, y_test, color=cmap(0.5), s=10)
29  plt.plot(X, y_pred_line, color='black', linewidth=2, label='Prediction')
30  plt.show()
```

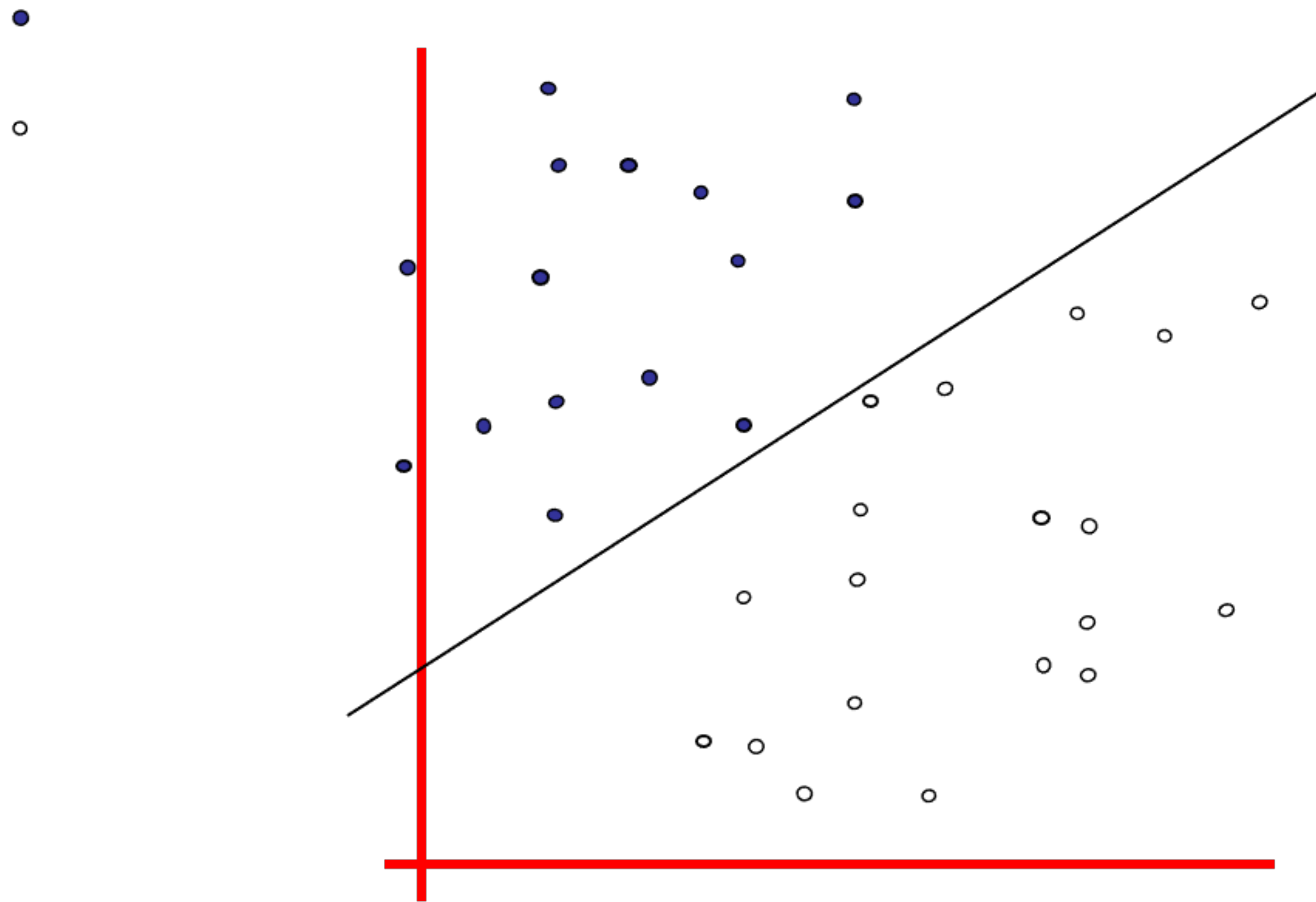
Linear Classifier

- denotes +1
- denotes -1



How would you
classify this data?

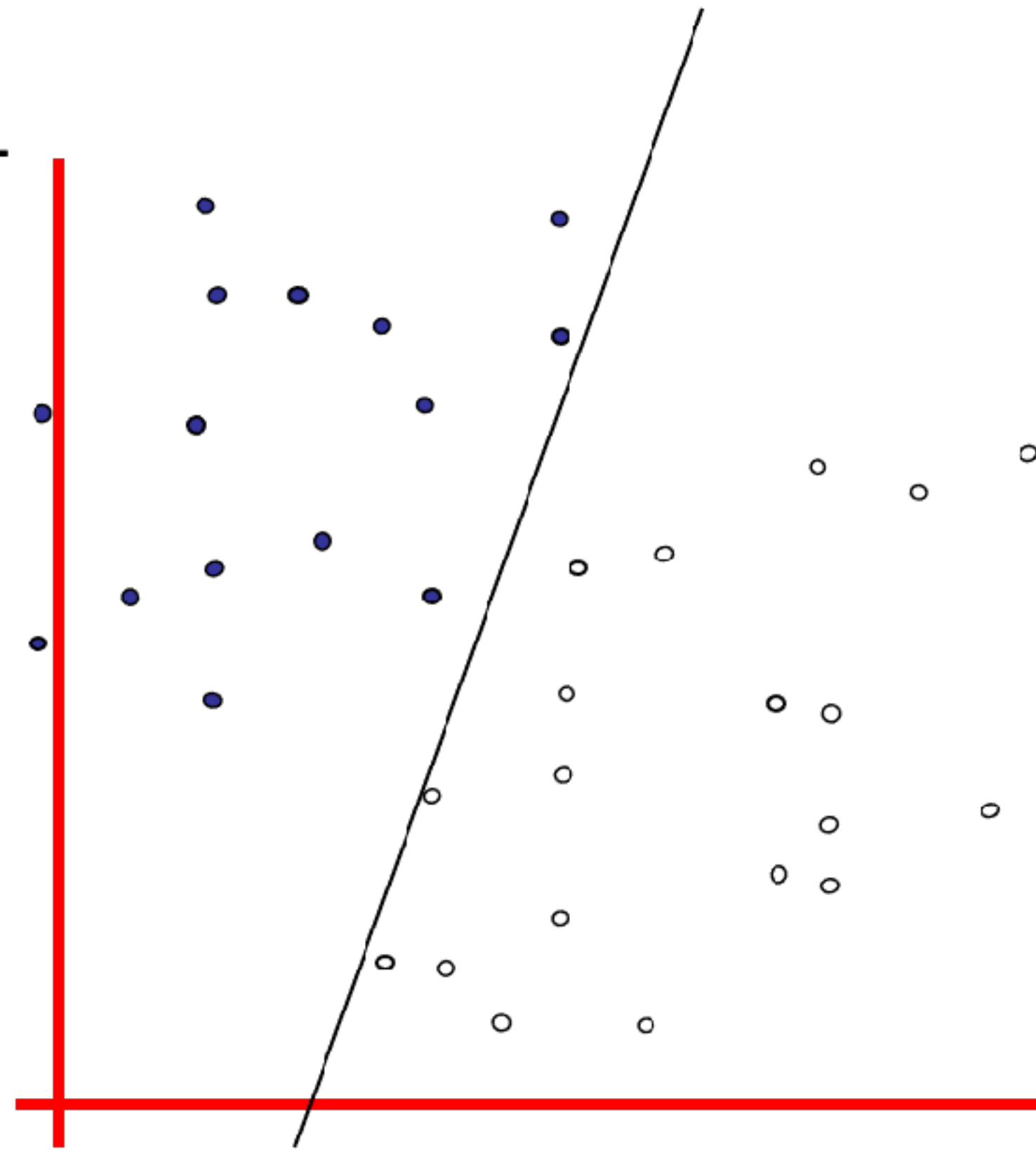
Linear Classifier



How would you
classify this data?

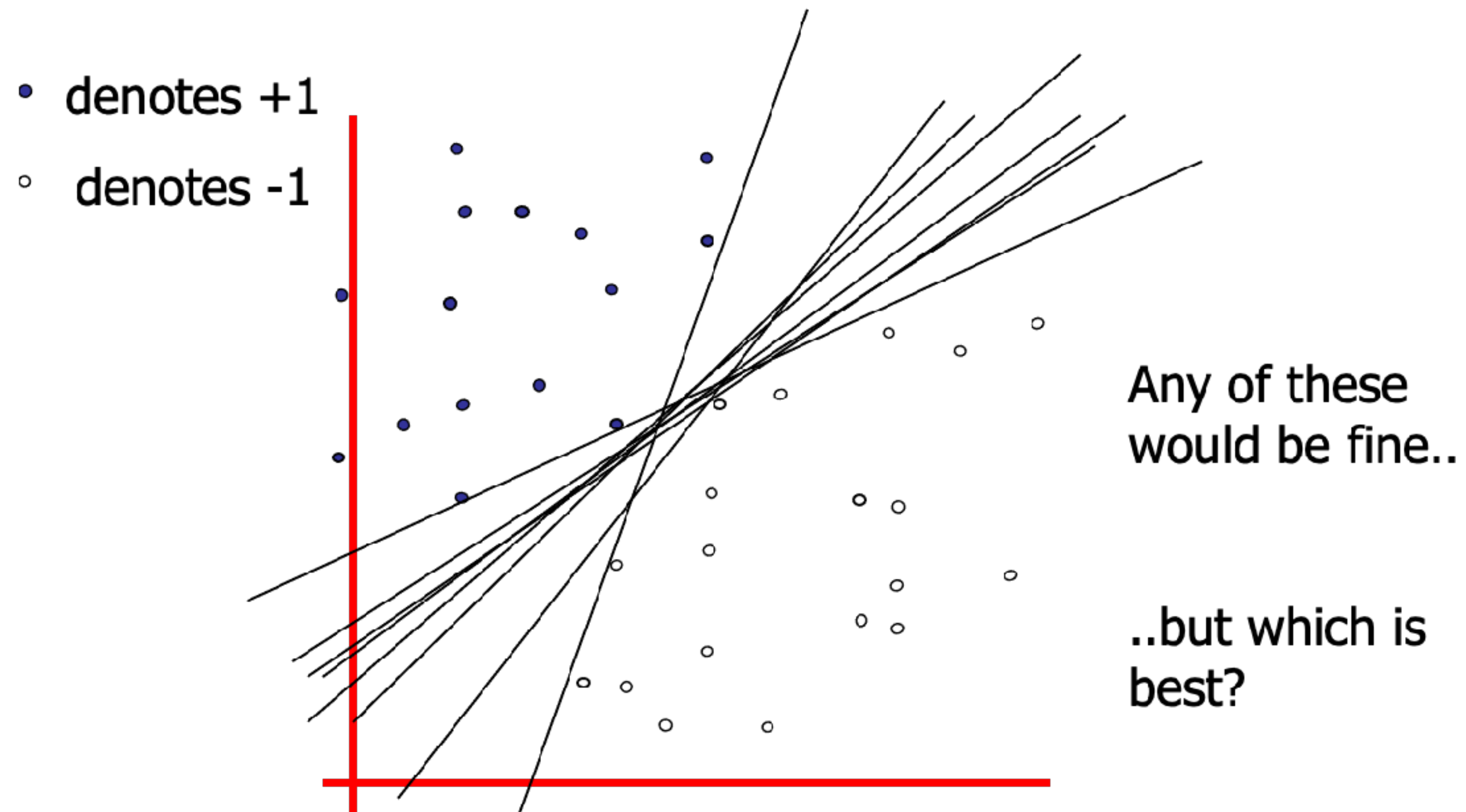
Linear Classifier

- denotes +1
- denotes -1



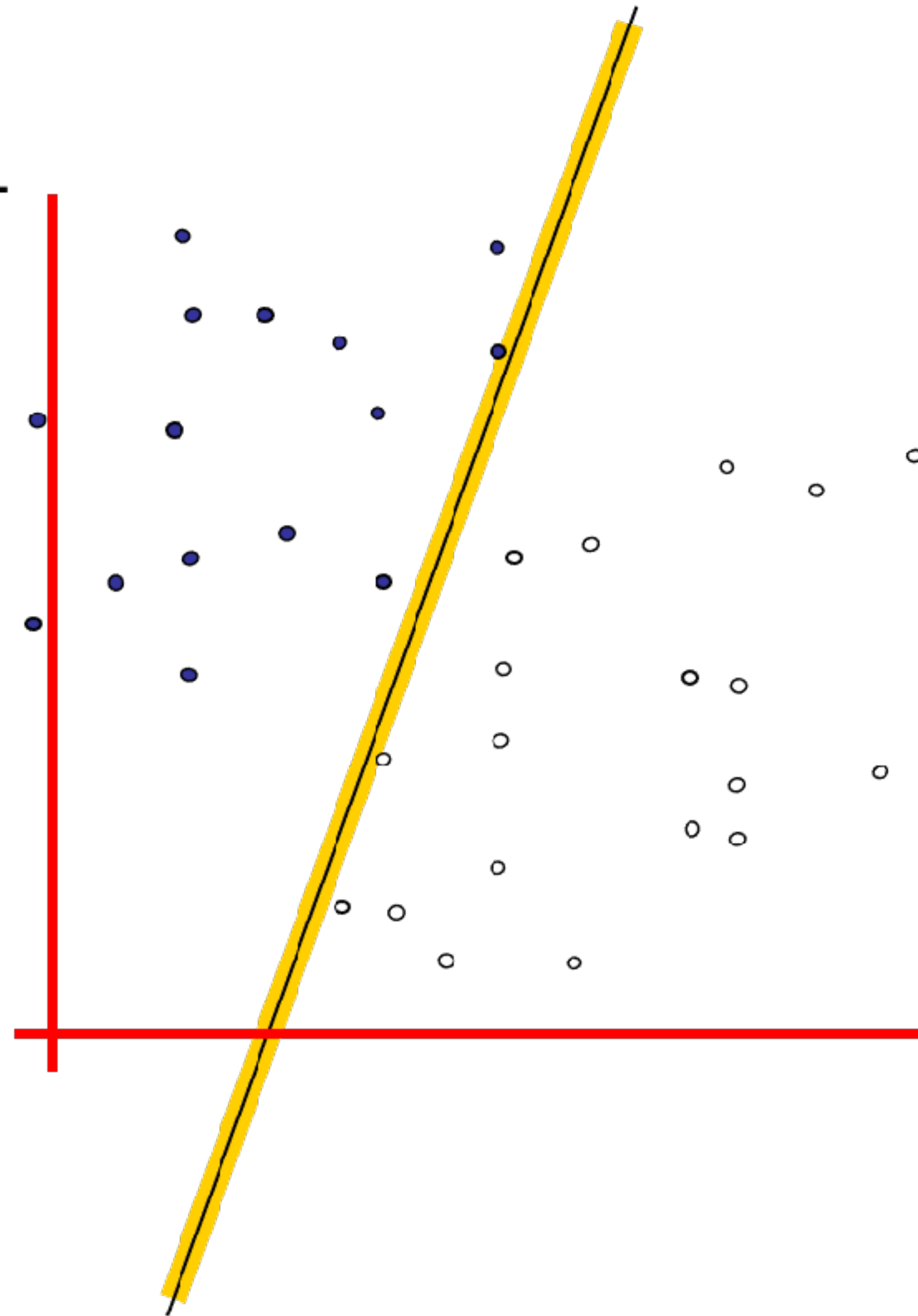
How would you
classify this data?

Linear Classifier



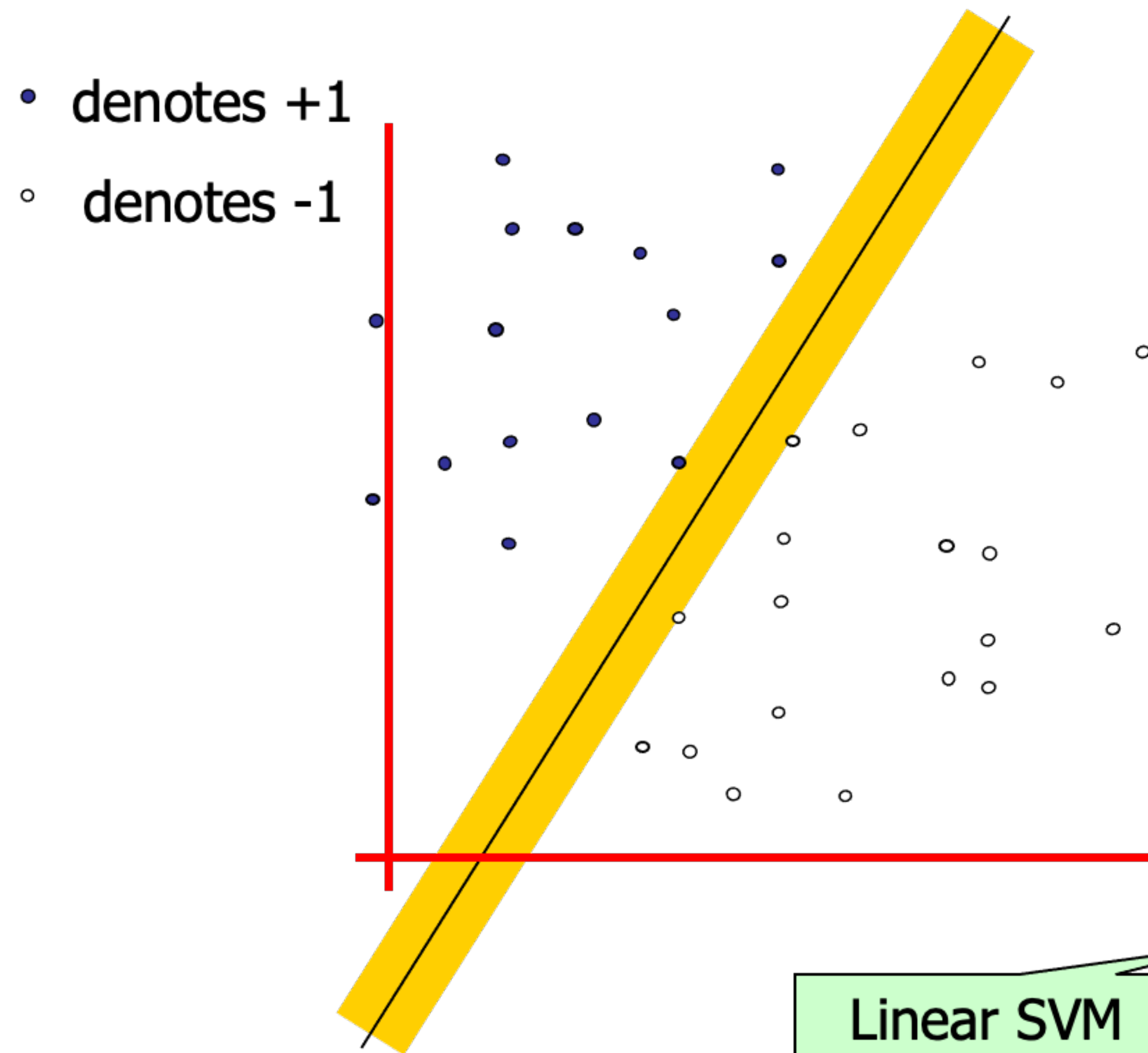
Classifier with Margin

- denotes +1
- denotes -1



Define the **margin** of a linear classifier as the width that the boundary could be increased by before hitting a datapoint.

Classifier with Maximum Margin



The **maximum margin linear classifier** is the linear classifier with the, maximum margin.

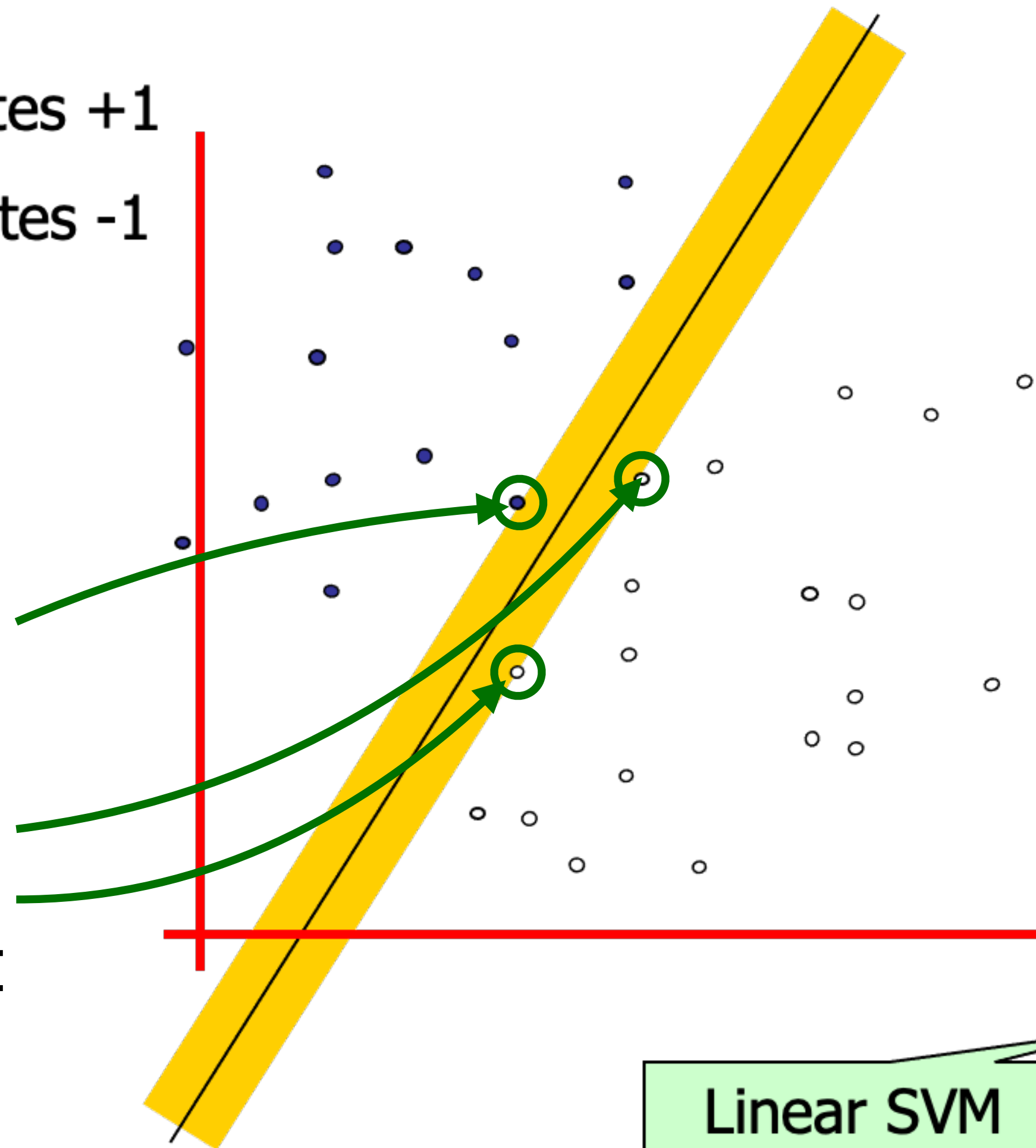
This is the simplest kind of **SVM** (called an **LSVM**)

Classifier with Maximum Margin

Intuitively **safe**, least chance of **misclassification**

Support Vectors
are those
datapoints that
the margin
pushes up against

- denotes +1
- denotes -1



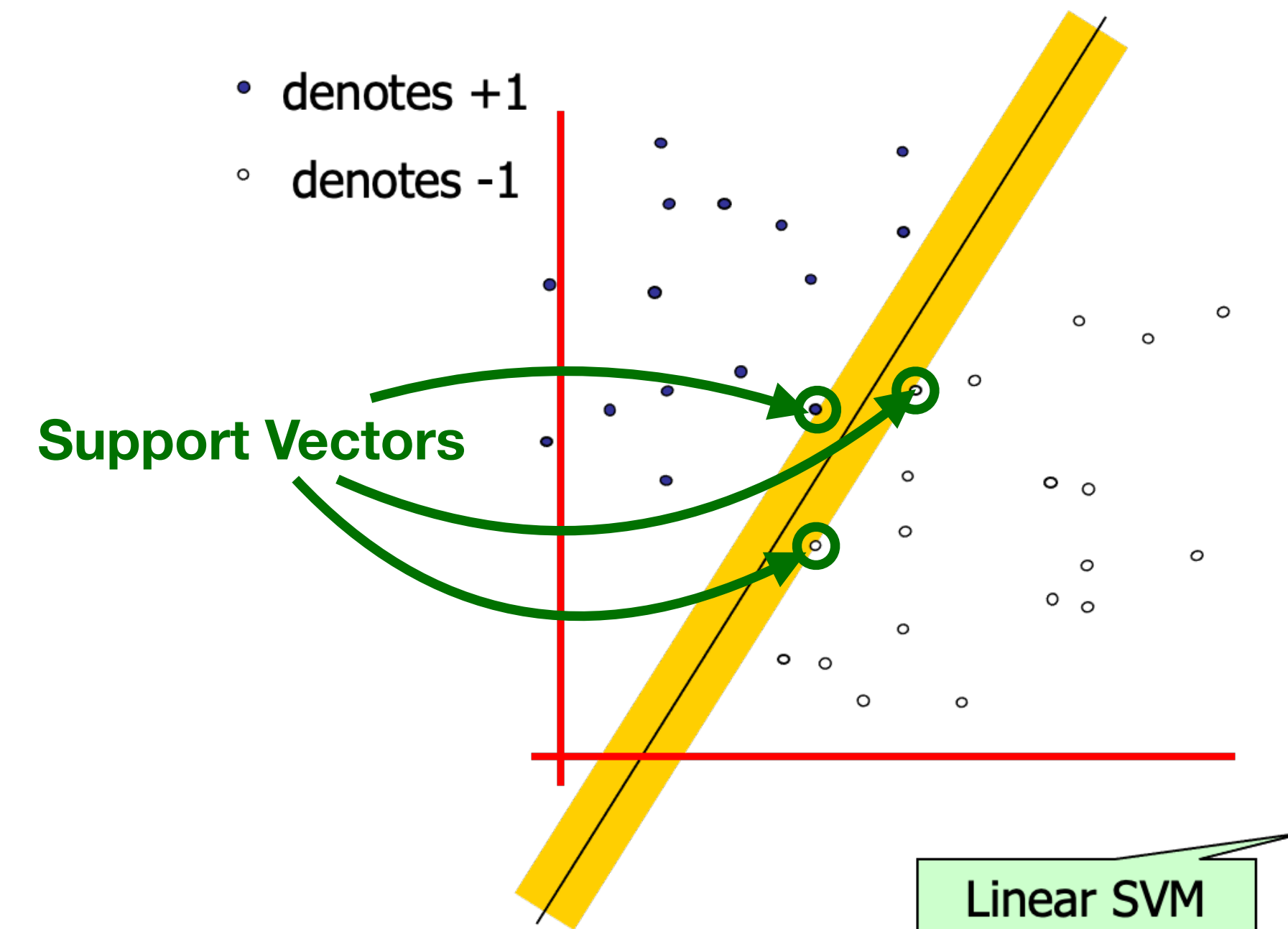
The **maximum margin linear classifier** is the linear classifier with the, maximum margin.

This is the simplest kind of **SVM** (called an **LSVM**)

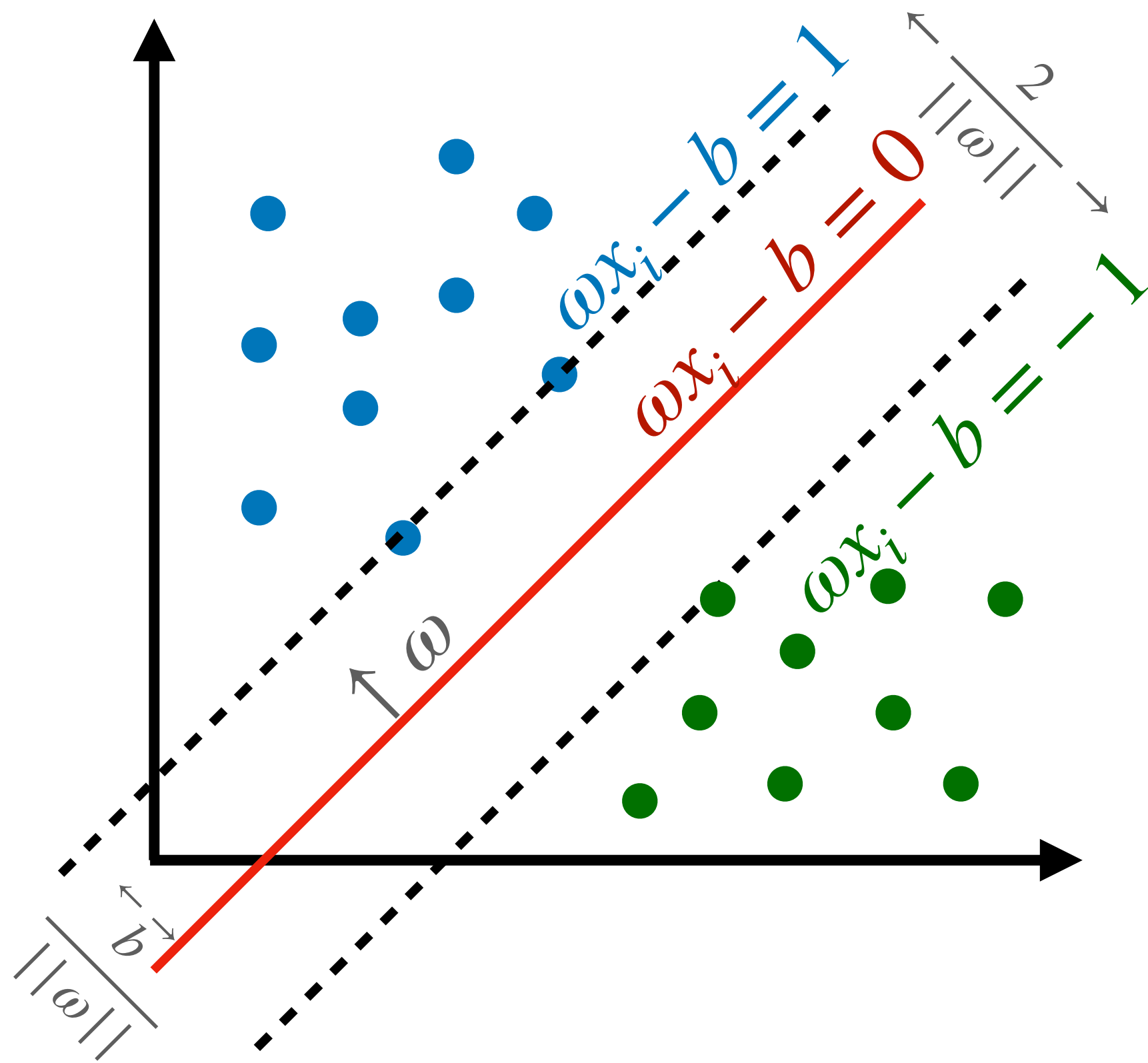
Support Vector Machine

Idea

- Finds a **linear boundary** that best separates the data. Linear boundary could be line, plane or hyperplane
- **The best hyperplane:**
 - is the one that yield the **largest separation/margin** between both classes
 - Is the one whose **distance** from the **nearest data point** on each side is maximized



Linear SVM



$$\omega \cdot x_i - b \geq 1 \quad \text{if } y_i = 1$$

$$\omega \cdot x_i - b \leq -1 \quad \text{if } y_i = -1$$

$$y_i(\omega \cdot x_i - b) \geq 1$$

$$y \in \{-1, 1\}$$

$$y = mx + b$$

$$y = x + 1$$

$$y - mx - b = 0$$

$$ax + by + c = 0$$

$$(a, b) \cdot (x, y) + c = 0$$

$$(a, b, c) \cdot (x, y, 1) = 0$$

$$-x + y - 1 = 0$$

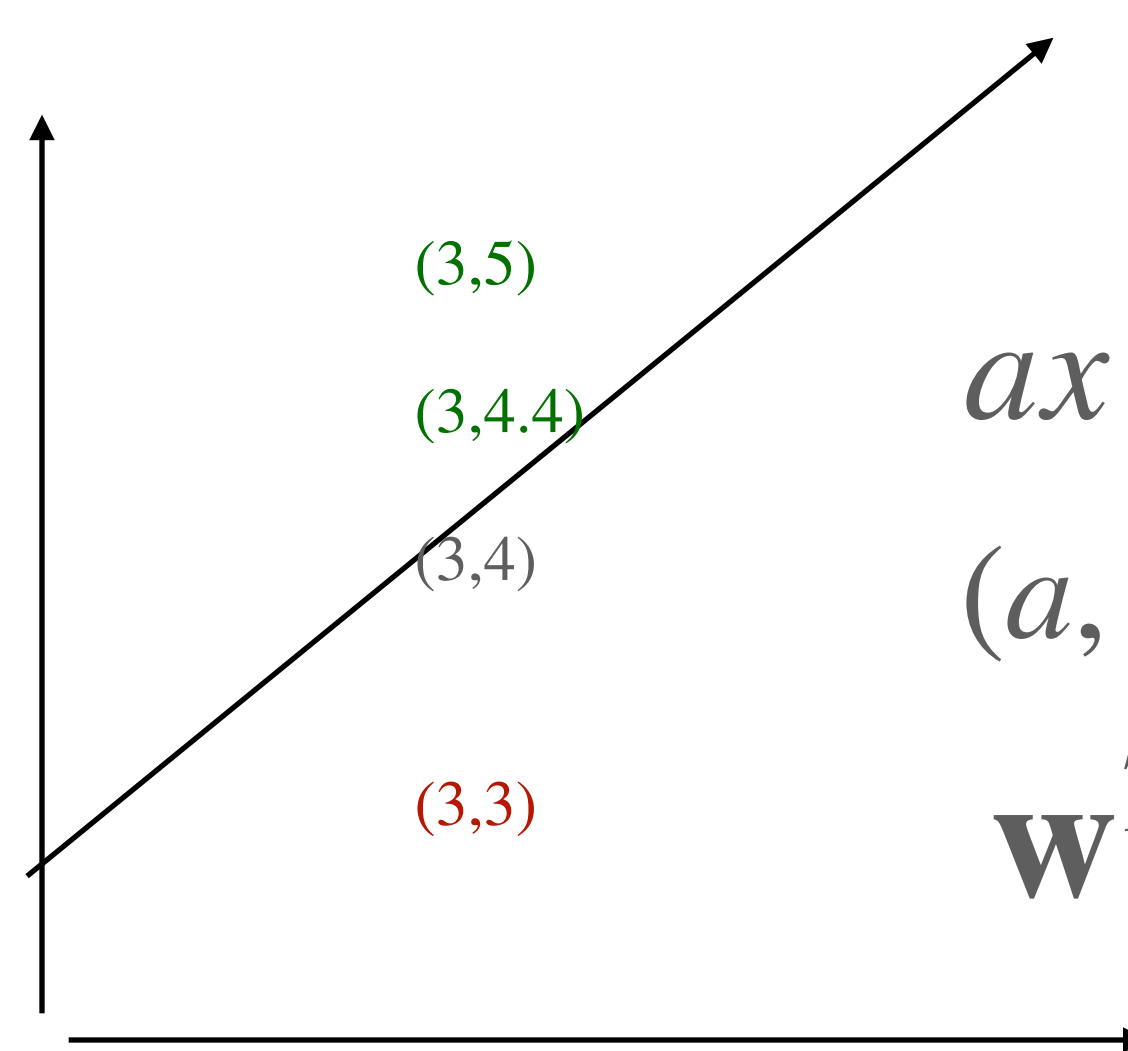
$$f(x, y) = -x + y - 1 = 0$$

$$f(3, 4) = -x + y - 1 = 0$$

$$f(3, 5) = -x + y - 1 = 1$$

$$f(3, 3) = -x + y - 1 = -1$$

$$f(3, 4.4) = -x + y - 1 = 0.4$$



$$ax + by + cz + d = 0$$

$$(a, b, c, d) \cdot (x, y, z, 1) = 0$$

$$\mathbf{w}^T \mathbf{x}$$

$$l = \max(0, 1 - y_i(\omega \cdot x_i - b))$$

$$l = \max(0, 1 - 0) = 1$$

$$l = \max(0, 1 - 1) = 0$$

$$l = \max(0, 1 - 1) = 0$$

$$l = \max(0, 1 - 0.4) = 0.6$$



Linear SVM

Hinge Loss

$$l = \max(0, 1 - y_i(\omega \cdot x_i - b))$$

Add Regularization

$$J = \lambda ||\mathbf{w}||_2^2 + \frac{1}{n} \max(0, 1 - y_i(\omega \cdot x_i - b))$$

maximize the margin

minimize the classification error

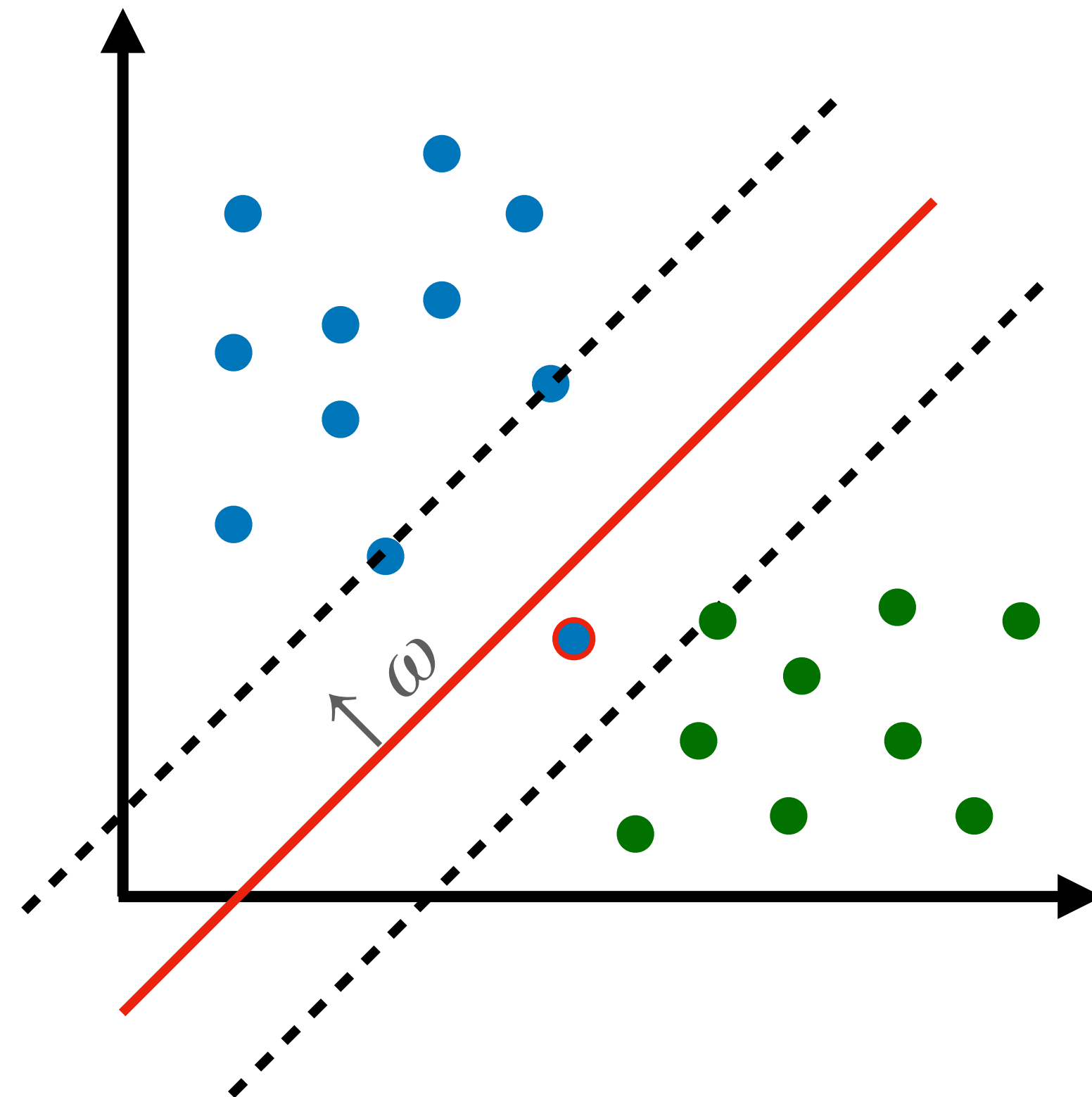
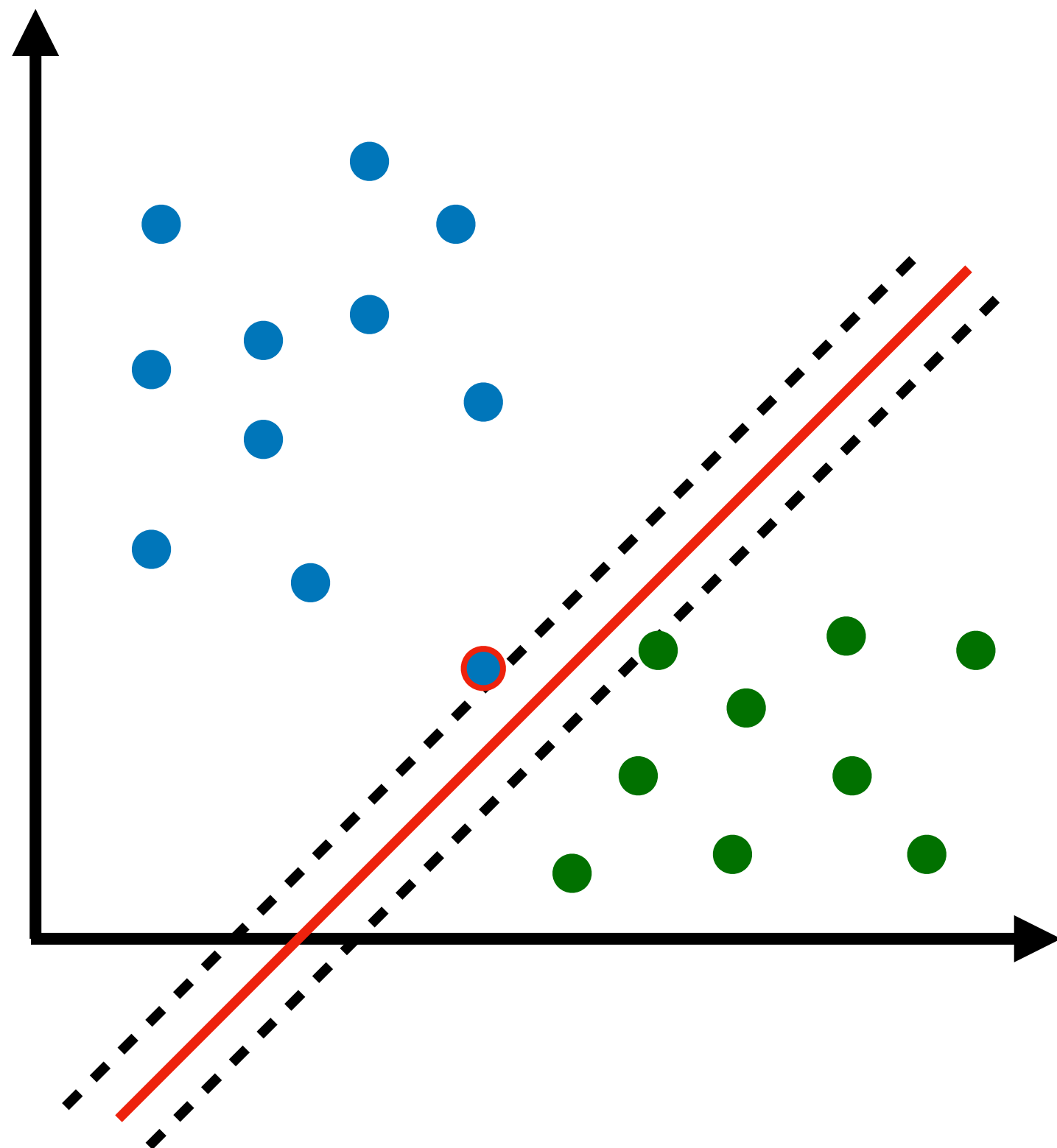
$$J = \lambda ||\mathbf{w}||_2^2$$

maximize
the margin

+

$$\frac{1}{n} \max(0, 1 - y_i(\omega \cdot x_i - b))$$

minimize the
classification error



Linear SVM

Hinge Loss

$$l = \max(0, 1 - y_i(\omega \cdot x_i - b))$$

Add Regularization

- $J = \lambda ||\omega||_2^2 + \frac{1}{n} \max(0, 1 - y_i(\omega \cdot x_i - b))$
- if $y_i \cdot f(x_i) \geq 1$:
 - $J_i = \lambda ||\omega||_2^2$
- else
 - $J_i = \lambda ||\omega||_2^2 + 1 - y_i(\omega \cdot x_i - b)$

Linear SVM - n dimensions

Hinge Loss

$$l = \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b))$$

$$\mathbf{w} = \{\omega_1, \omega_2 \cdots \omega_n\}^T$$

$$\mathbf{x} = \{x_1, x_2 \cdots x_n\}^T$$

Add Regularization

- $J = \lambda ||\mathbf{w}||_2^2 + \frac{1}{n} \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b))$

- if $y_i \cdot f(\mathbf{x}_i) \geq 1$:

- $J_i = \lambda ||\mathbf{w}||_2^2$

- else

- $J_i = \lambda ||\mathbf{w}||_2^2 + 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b)$

Linear SVM

Update Rule

$$J = \lambda ||\mathbf{w}||_2^2 + \frac{1}{n} \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b))$$

- if $y_i(\mathbf{x}_i \cdot \mathbf{w} - b) \geq 1$:

$$\omega_k = \omega_k - \eta \frac{\partial J}{\partial \omega_k} = \omega_k - \eta \cdot 2\lambda \omega_k$$

$$\mathbf{w} = \mathbf{w} - \eta \frac{\partial J}{\partial \mathbf{w}} = \mathbf{w} - \eta \cdot 2\lambda \mathbf{w}$$

$$J_i = \lambda ||\mathbf{w}||_2^2$$

$$\mathbf{w} = \{\omega_1, \omega_2 \cdots \omega_n\}^T$$

$$J_i = \lambda(\mathbf{w}^T \mathbf{w})$$

$$\mathbf{x} = \{x_1, x_2 \cdots x_n\}^T$$

$$J_i = \lambda(\{\omega_1, \omega_2 \cdots \omega_n\}^T \{\omega_1, \omega_2 \cdots \omega_n\})$$

$$J_i = \lambda(\omega_1^2 + \omega_2^2 + \omega_3^2 + \cdots \omega_n^2)$$

$$\frac{\partial J}{\partial \omega_i} = \frac{\partial J}{\partial w_i}(\lambda(\omega_1^2 + \omega_2^2 + \omega_3^2 + \cdots \omega_n^2))$$

$$\frac{\partial J}{\partial \omega_2} = \frac{\partial J}{\partial w_2}(\lambda(\omega_1^2 + \omega_2^2 + \omega_3^2 + \cdots \omega_n^2)) = 2\lambda\omega_2$$

Linear SVM

Update Rule

$$J = \lambda ||\mathbf{w}||_2^2 + \frac{1}{n} \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b))$$



- if $y_i(\mathbf{x}_i \cdot \mathbf{w} - b) \geq 1$:

$$\omega_k = \omega_k - \eta \frac{\partial J}{\partial \omega_k} = \omega_k - \eta \cdot 2\lambda \omega_k$$

$$\mathbf{w} = \mathbf{w} - \eta \frac{\partial J}{\partial \mathbf{w}} = \mathbf{w} - \eta \cdot 2\lambda \mathbf{w}$$

- else:

$$\omega_k = \omega_k - \frac{\partial J}{\partial \omega_k} = \omega_k - \eta \cdot (2\lambda \omega_k - y_i \cdot x_i)$$

$$\mathbf{w} = \mathbf{w} - \frac{\partial J}{\partial \mathbf{w}} = \mathbf{w} - \eta \sum_{i=1}^n (2\lambda \mathbf{w} - y_i \cdot \mathbf{x}_i)$$

$$b = b - \eta \frac{\partial J}{\partial b} = b - \sum_{i=1}^n \eta \cdot y_i$$

Linear SVM

Steps



- **Training** (learn weights):
 - Initialize weights
 - Make sure $y \in \{-1, 1\}$
 - Apply update rules of n_iters
- **Prediction:**
 - Calculate $y = \text{sign}(\mathbf{w}^T \mathbf{x} - b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$

Linear SVM

Python Implementation

```
fit(X,y)
```

```
n_samples = rows of X
```

```
n_features = columns of X
```

```
w = zeros(n_features,1)
```

```
b = 0
```

```
y = if y ≤ 0, -1, otherwise 1
```

```
for each iteration upto n_iters  
  for each sample  $\mathbf{x}_i$  in X
```

- if $y_i(\mathbf{x}_i \cdot \mathbf{w} - b) \geq 1$:

$$\mathbf{w} = \mathbf{w} - \eta \cdot 2\lambda \mathbf{w}$$

- else:

$$\mathbf{w} = \mathbf{w} - \eta \cdot (2\lambda \mathbf{w} - y_i \cdot \mathbf{x})$$

$$b = b - \eta \cdot y_i$$

- **Training** (learn weights):
 - Initialize weights
 - Make sure $y \in \{-1, 1\}$
 - Apply update rules of *n_iters*

SVM Init

η - learning rate

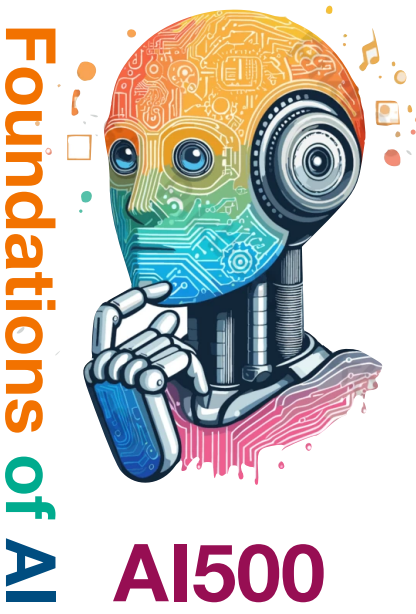
λ

n_iters

$\mathbf{w}$, b

Linear SVM

Python Implementation

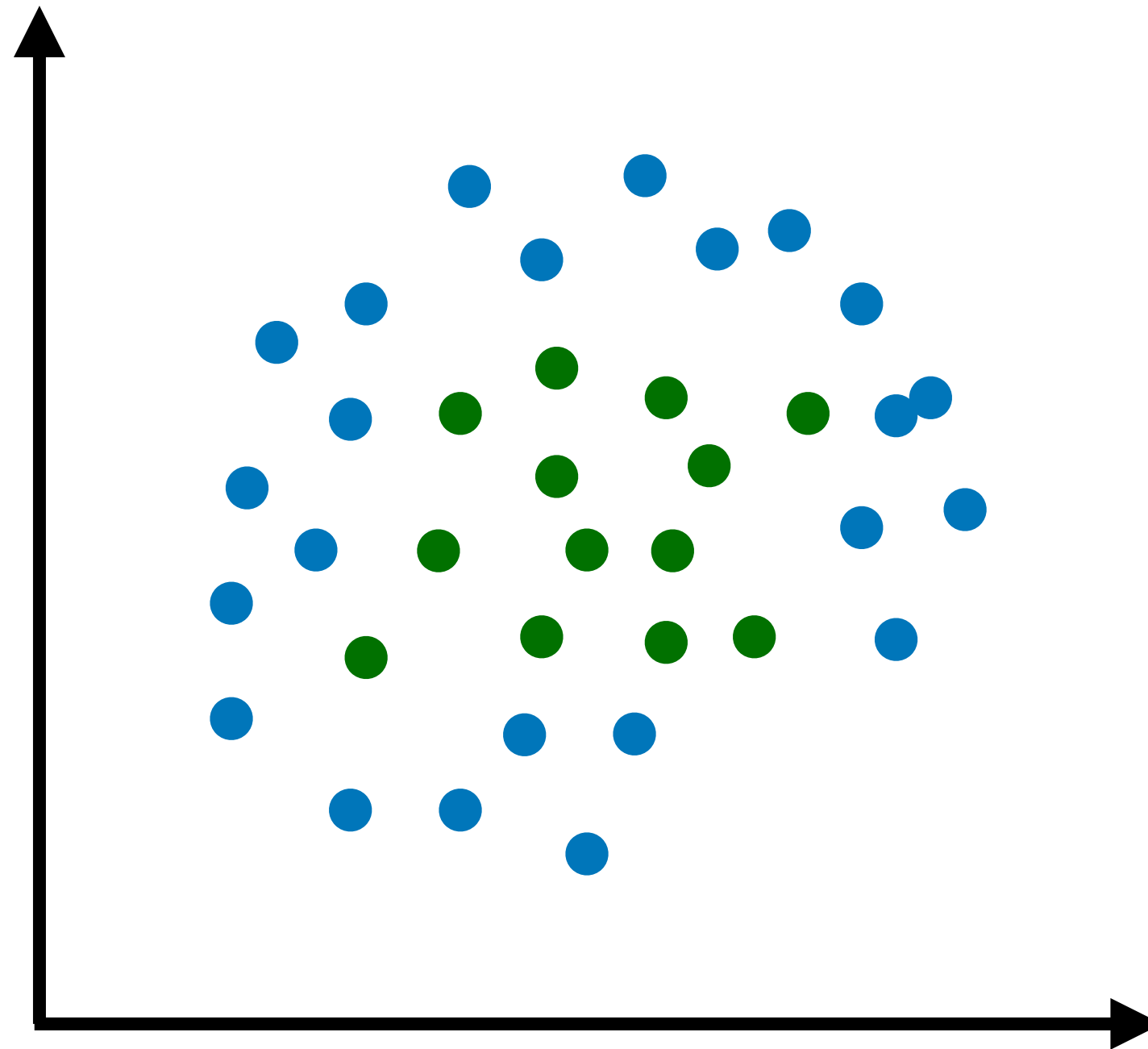
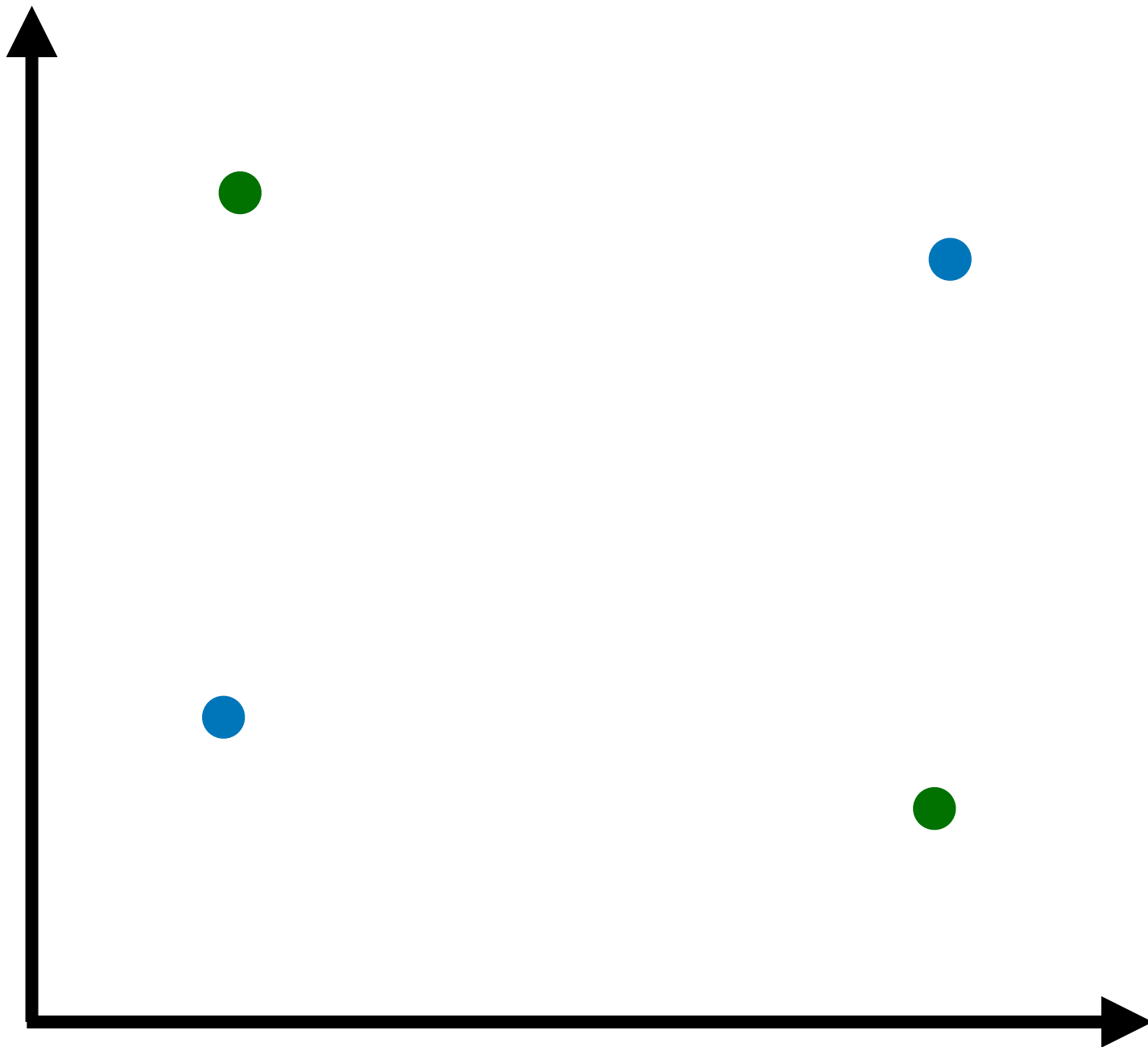


- Prediction:
 - Calculate
$$y = \text{sign}(\mathbf{w}^T \mathbf{x} - b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

`predict (x)`
$$y = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

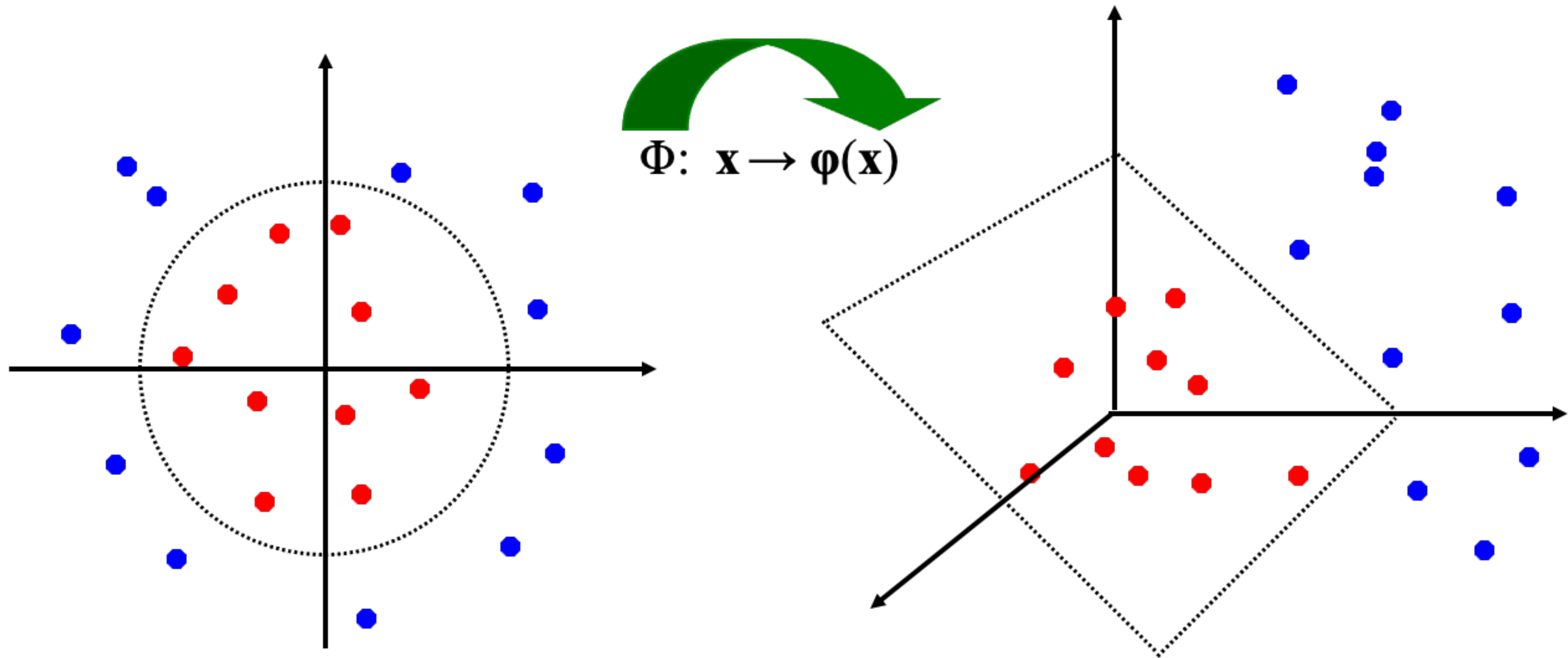
SVM

Data with non-linear decision boundary



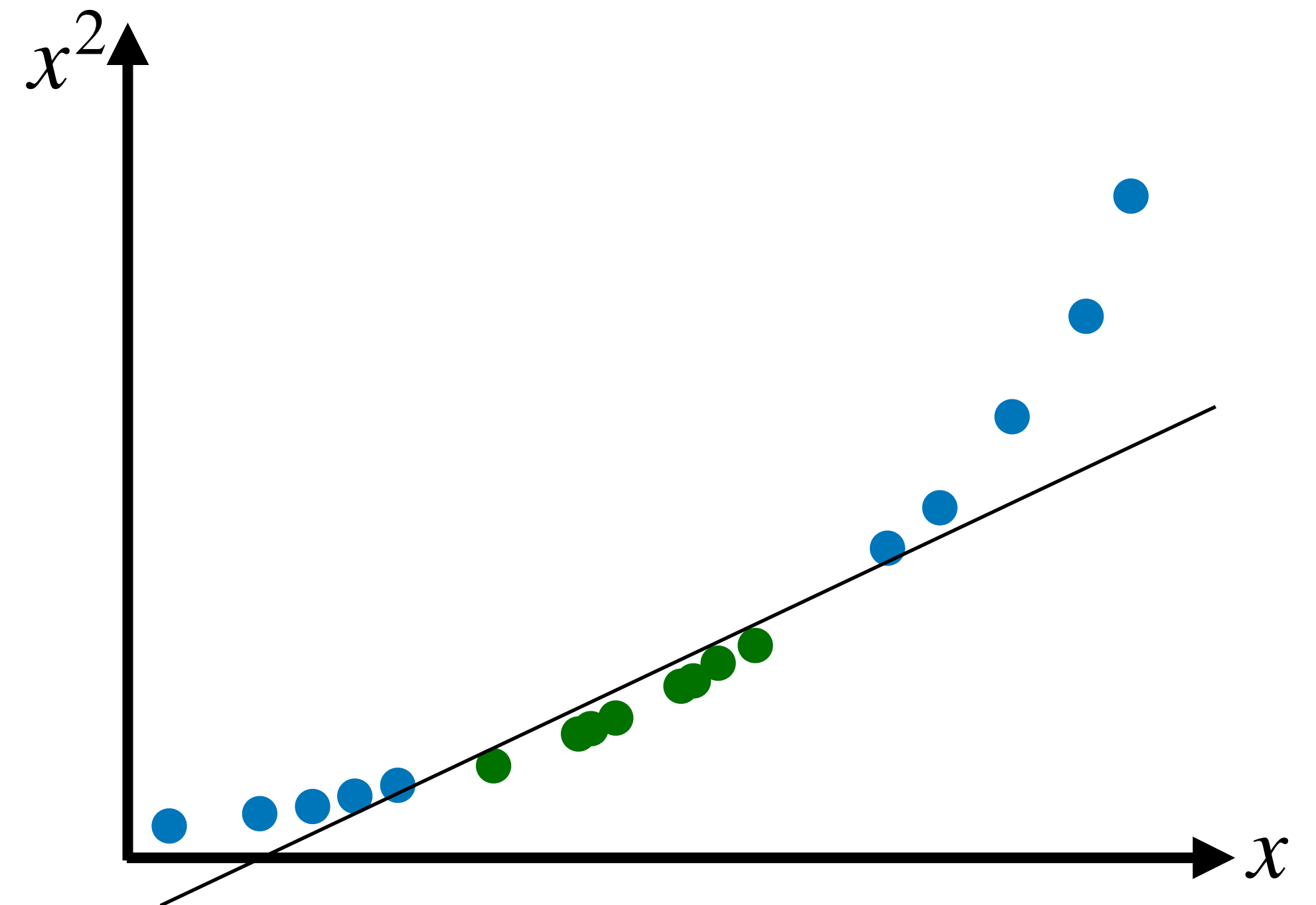
SVM

Data with non-linear decision boundary - Kernel Trick



SVM

Data with non-linear decision boundary - Kernel Trick



SVM

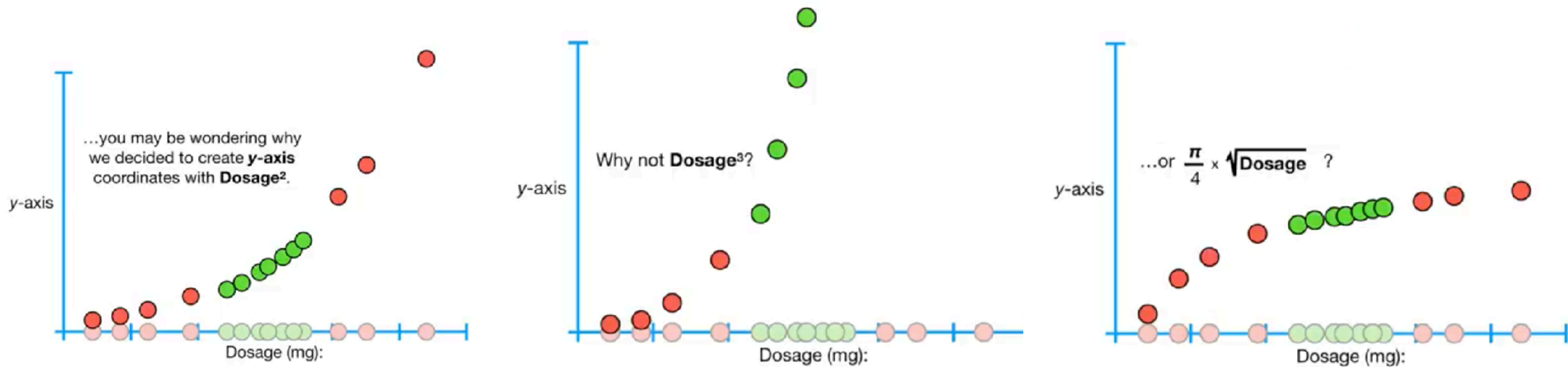
Types of Kernel

- Polynomial
- Radial Basis Function
- Sigmoid
- Quadratic functions
- ...



SVM

Kernel Trick



How to decide how to transform the data?

SVM

Kernel Trick

- The linear classifier relies on **inner product between vectors** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- If every **datapoint** is mapped into **high-dimensional space** via some transformation $\Phi : \mathbf{x} \rightarrow \phi(\mathbf{x})$, the inner product becomes:
$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$
- A **kernel function** is a function that is equivalent to an **inner product in some feature space**.

Polynomial Kernel

- $\mathbf{x} = \{x_1, x_2, \dots, x_k\}$, Let $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + \boxed{r})^{\boxed{d}}$

Coefficients

Degree

SVM

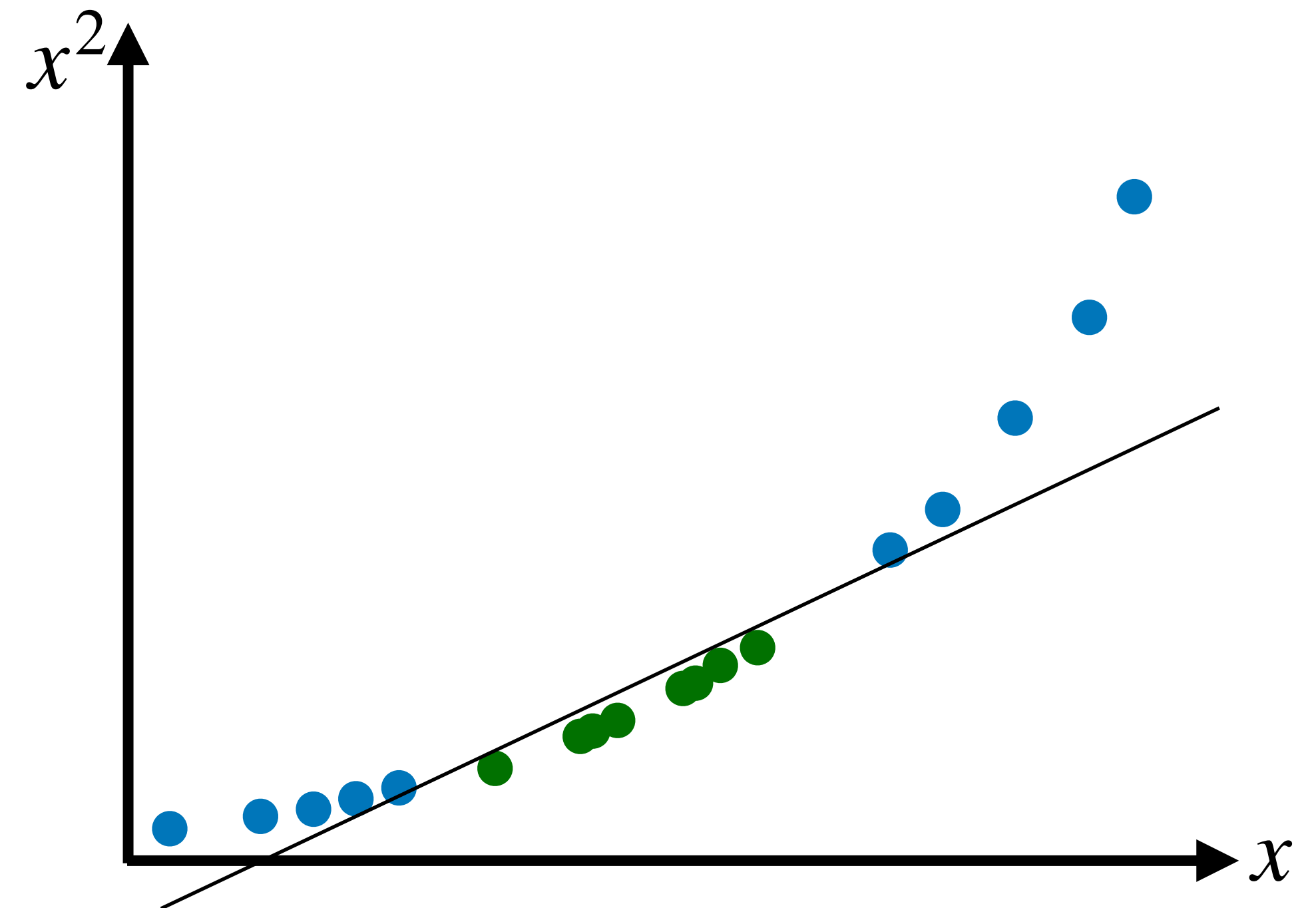
Kernel Trick

Polynomial Kernel

- $\mathbf{x} = \{x_1, x_2, \dots, x_k\}$; Let $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + r)^d$
Coefficients Degree

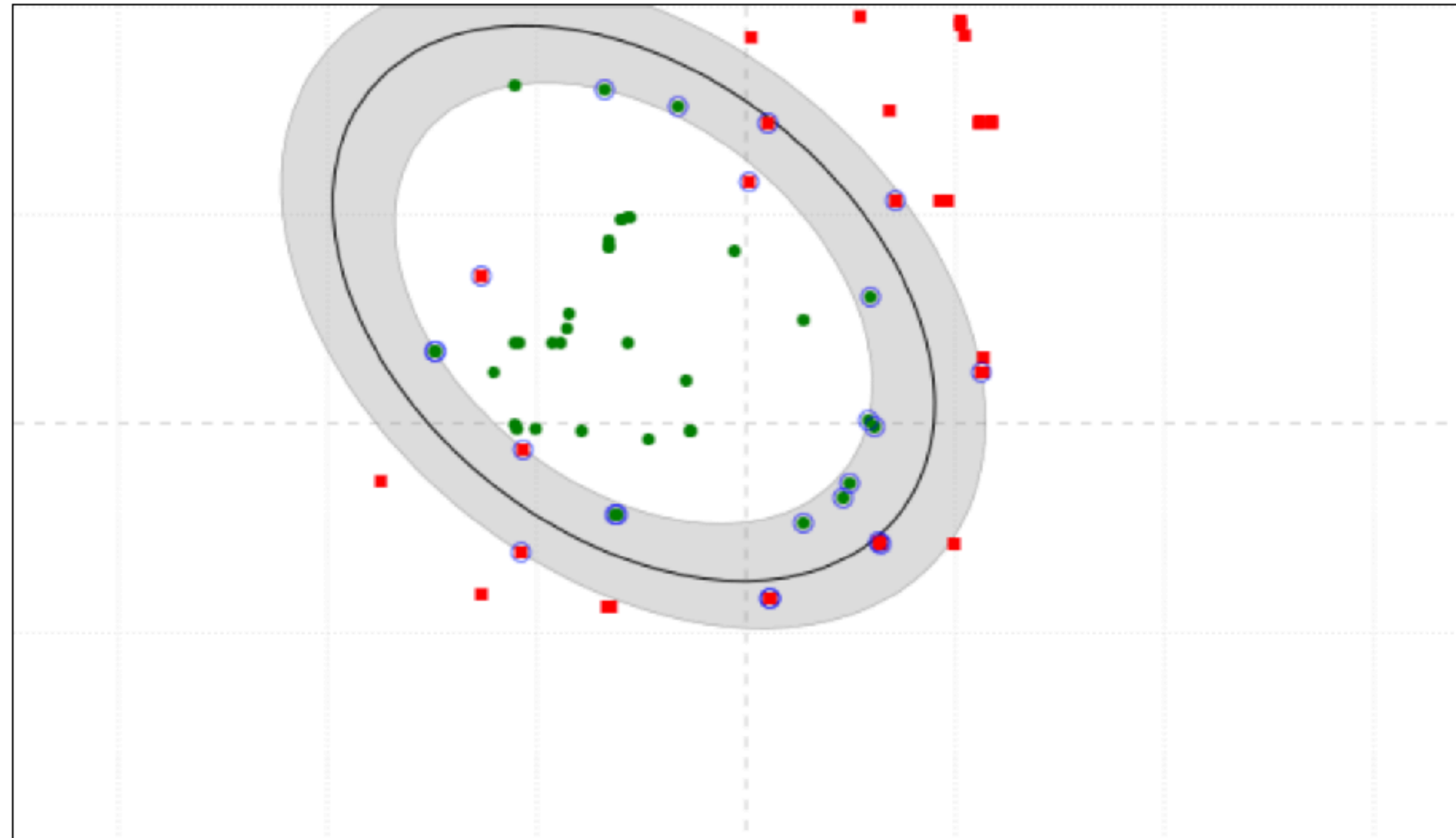
2D Example

- $\mathbf{x} = \{x_1, x_2\}$;
- Let $r = 1$, $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + 1)^2$
- $K(\mathbf{x}_i, \mathbf{x}_j) = 2\mathbf{x}_i^T \mathbf{x}_j + (\mathbf{x}_i^T \mathbf{x}_j)^2 + 1$
- $K(\mathbf{x}_i, \mathbf{x}_j) = (\sqrt{2}\mathbf{x}_i, \mathbf{x}_i^2, 1)^T (\sqrt{2}\mathbf{x}_j, \mathbf{x}_j^2, 1)$
- $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$



SVM Demo

<https://greitemann.dev/svm-demo>



☒ Toggle $\nu = 0.28$

Kernel: Quadratic

$\gamma = 1.0$ $c_0 = 6$

$$K(\mathbf{x}, \mathbf{y}) = (\gamma \mathbf{x} \cdot \mathbf{y} + c_0)^2$$

☒ Highlight support vectors

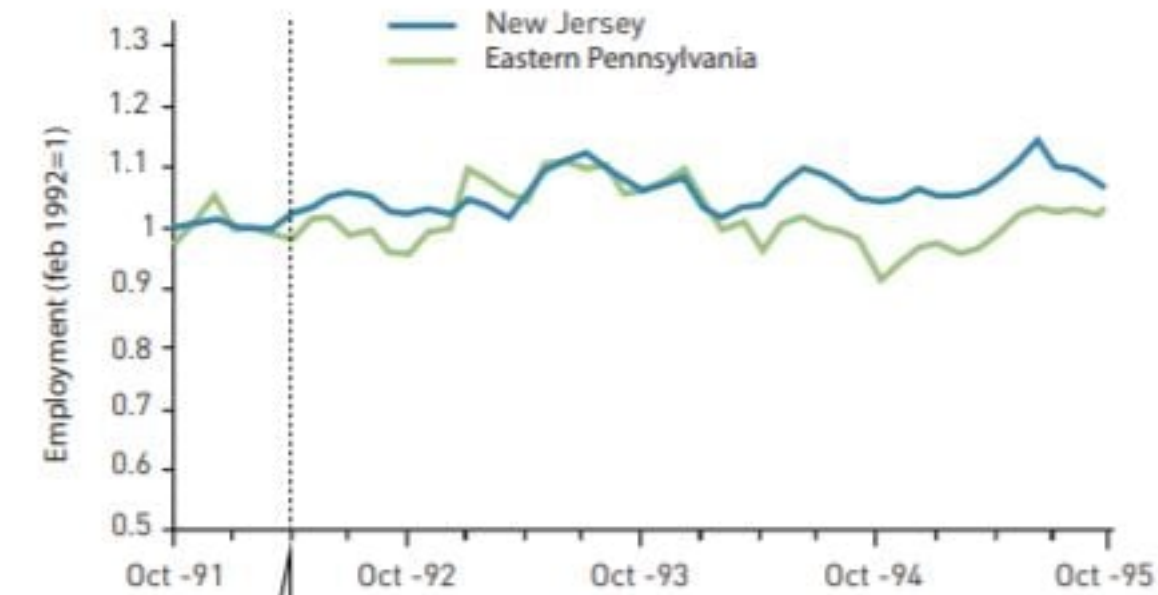
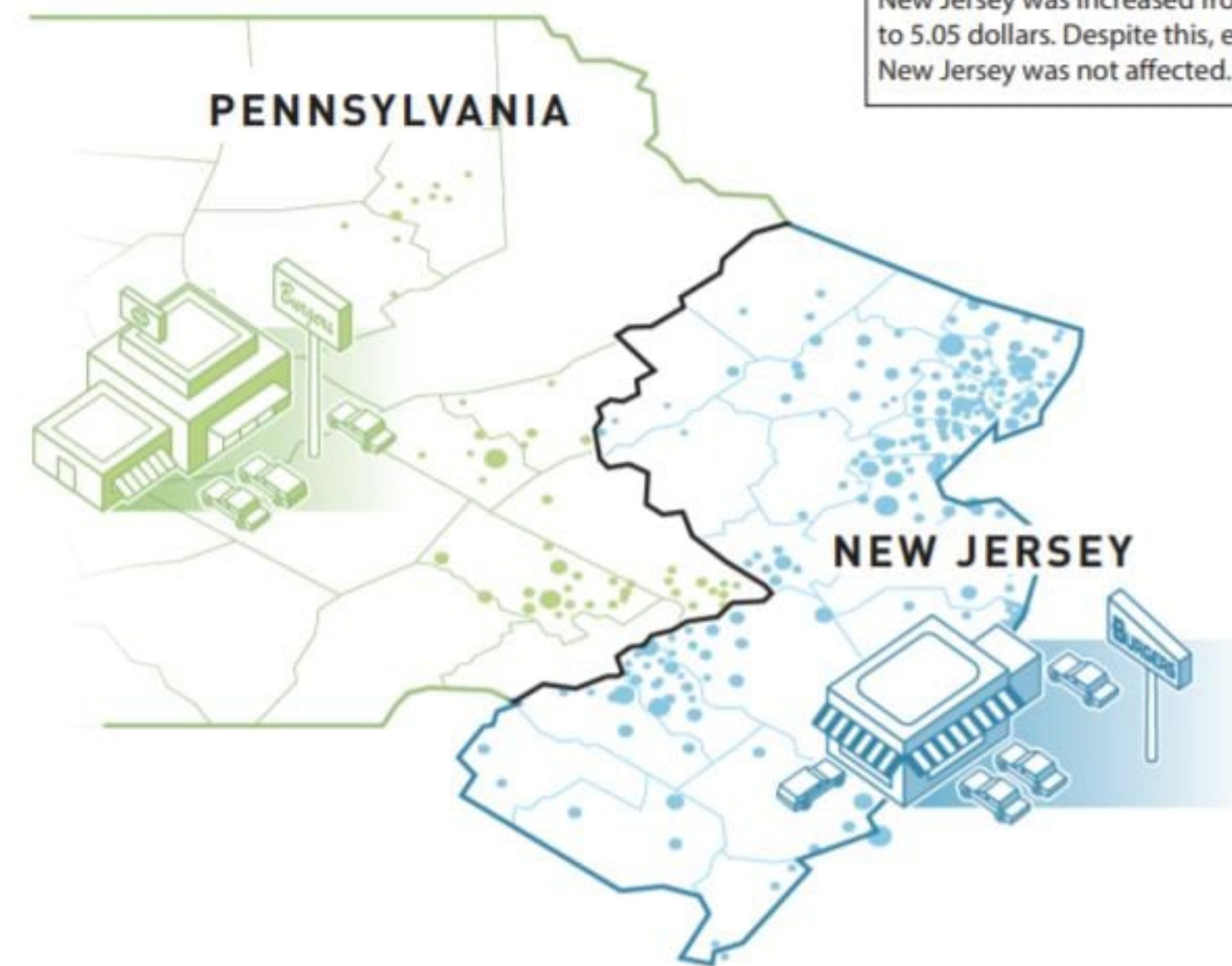
Next...Causal Inference

The effect of increasing the minimum wage

Card and Krueger used a natural experiment to study how increasing the minimum wage affects employment.

The researchers identified a treatment group (restaurants in New Jersey) and a control group (restaurants in eastern Pennsylvania) to measure the effect of increasing the minimum wage.

● CONTROL GROUP ● TREATMENT GROUP



1 April 1992: The hourly minimum wage in New Jersey was increased from 4.25 dollars to 5.05 dollars. Despite this, employment in New Jersey was not affected.

Source: ©Johan Jarnestad/The Royal Swedish Academy of Sciences